COSC419/519:DEEP LEARNING

Recurrent Neural Networks (RNN)

Dr. Mohamed Shehata

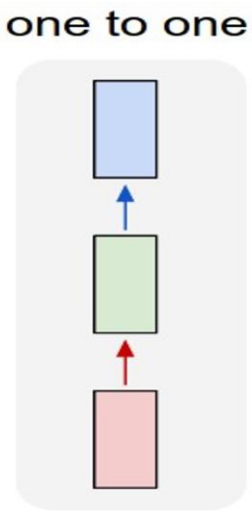
Computer Science

University of British Columbia

UBC OKANAGAN

The Problem of Sequential Data

- Standard Feed-Forward NNs assume inputs and outputs are fixed-size tensors
- In reality, data can be sequential and variable in length:
  - NLP: Sentences of varying word counts
  - Time-Series: Stock prices over arbitrary timeframes
- Core failure of FFNNs: Lack of memory
  - Processing x_t happens independently of x_{t-1}
  - This means the model is memoryless, i.e. it has no memory of anything before.
    - sometimes long-distance context can be important such as in the context of NLP (you need the past words to understand the current one)



Language Modeling (LM)

- Language Modeling is foundational task of predicting the next word.
- Given a sequence of words: $x_1, \dots, x_t$
  - Task: Compute the probability distribution of the next word x_{t+1}
  - Mathematically: $P(x_{t+1} | x_1, \dots, x_t)$
- Vocabulary V is a predefined finite set of discrete tokens.
- Any sequence probability can be factored completely:
  - To evaluate the joint likelihood of an entire sentence, we multiply conditional probabilities.

$$P(x_1, \dots, x_T) = \prod_{t=1}^{T} P(x_t \mid x_1, \dots, x_{t-1})$$

- Thus, an accurate LM allows us to score the fluency of any sequence.

Evaluating LMs: Perplexity

- In NLP, we evaluate Language Models using Perplexity.
- Perplexity is the inverse probability of the corpus, normalized by the number of words.
  - **corpus** is a large, structured dataset of text or spoken language (e.g., the complete works of Shakespeare)
- It is equivalent to the exponentiation of the Cross-Entropy loss:

$$\text{Perplexity} = \exp\big(J(\theta)\big)$$

where $J(\theta)$: The loss function (typically cross-entropy) parameterized by θ

- Lower perplexity indicates the model is less 'confused' by test data.

Traditional Approach: n-gram LMs

- An n-gram is a consecutive chunk of n tokens.
- To simplify, we make a Markov assumption:
  - x_n depends ONLY on the preceding $(n-1)$ words.
- Example (4-gram LM): $P(w_4|w_1, w_2, w_3)$.
- We calculate probabilities via Maximum Likelihood Estimation (MLE) based on raw corpus counts.

Transitioning to Neural Language Models

- Idea: Learn distributed, dense representations (embeddings) of words.
- Instead of exact string matching, neural nets map tokens to a continuous geometric space.
- This solves the sparsity problem: similar words yield similar vectors.
- More on this when we talk about attention and transformers

Fixed-Window Neural LM

(Bengio et al., 2003)

- Input: A fixed window of previous words (e.g., 4 words).
- Step 1: Convert each word to dense vector embedding:

$$\boldsymbol{e} = [\boldsymbol{e_1}; \boldsymbol{e_2}; \boldsymbol{e_3}; \boldsymbol{e_4}]$$

- Step 2: Feed through non-linear hidden layer:

$$\boldsymbol{h} = \boldsymbol{f}(\boldsymbol{W} * \boldsymbol{e} + \boldsymbol{b_1})$$

- Step 3: Output layer with softmax:

$$\hat{y} = \boldsymbol{softmax}(\boldsymbol{U} * \boldsymbol{h} + \boldsymbol{b_2})$$

U is the weight matrix of the output layer and h is hidden layer

- Output: A dense probability distribution over $|V|$

The Asymmetry Problem

- The "Fixed-Window" approach is inefficient: it treats the exact same word differently just because it appears in a different spot in the sentence. This is known as The Asymmetry Problem.
- In this model, the large weight matrix W is essentially chopped into sections (sub-matrices).
  - Position 1 (e_1) is multiplied by $W^{(1)}$.Position 2 (e_2) is multiplied by $W^{(2)}$. Even if the word "apple" appears in both positions, the math used to process it is completely different because $W^{(1)}$ and $W^{(2)}$ are separate sets of numbers.
- There is NO parameter sharing across time steps.
- The window is rigidly fixed.
  - If the subject of a sentence was mentioned 10 words ago, and window size is 5, the model has literally zero mathematical access to the subject.
  - Expanding the window size substantially increases parameter counts.

Core Design Requirements for Sequence Models

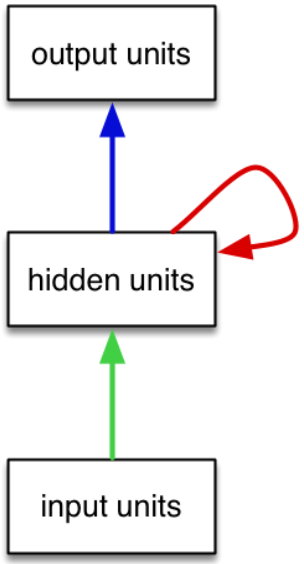
- To process language properly, our architecture MUST possess:
  - Ability to handle variable-length sequences dynamically.
  - Capacity to track long-term dependencies (arbitrary history size).
  - Maintenance of sequential order.
  - Parameter sharing across the sequence (position-invariant rules).

RNN

Recurrent Neural Network (RNN)

- The RNN is an architecture designed specifically to fulfill all sequence requirements.
- Core concept: Introduce a Hidden State h_t that evolves over time.
- The network applies a continuous recurrence formula at every discrete time step.
- Crucially, the exact same weights are reused at every single step $\boldsymbol{t}$.

The Concept of the Hidden State



Key idea: RNNs have an "internal state" that is updated as a sequence is processed

- At every time step t, the RNN computes a hidden state vector h_t
- h_t is a function of the current input x_t AND the previous hidden state h_{t-1}
- Therefore, h_t theoretically contains information from $\mathrm{x}_t, \mathrm{x}_{t-1}, \ldots,$ back to x_1
- h_t acts as the network internal short-term memory

Vanilla RNN: Formal Equations

- **At each timestep t, the network computes hidden state h_t and output $\hat{y}_t$**

new state ← $h_t = f_w(h_{t-1}, x_t)$

a function with parameters W

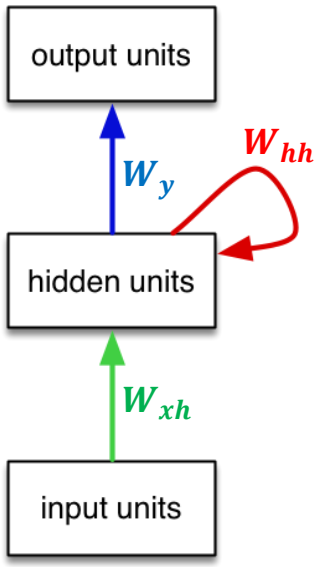
Old state

input vector at time step t

output ← $\hat{y}_t = f_{w_y}(h_t)$

another function with parameters w_y

new state



Example:

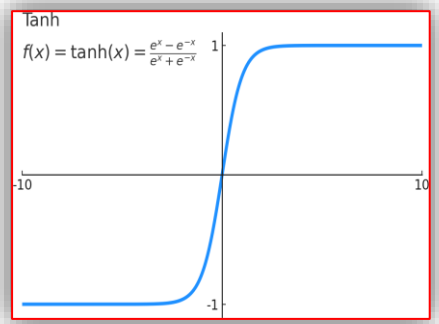
$$h_t = tanh(W_{hh} * h_{t-1} + W_{xh} * x_t + b_h)$$
$$\hat{y}_t = softmax(W_y * h_t + b_y)$$

Sometimes called a "Vanilla RNN" or an "Elman RNN" after Prof. Jeffrey Elman

x_t: Input vector at time t.

h_t: Hidden state vector representing accumulated context.

W_{hh}, W_{xh}, W_y: Learnable shared weight matrices.



Dimensionality of RNN Parameters

- **Let d = embedding size, D_h = hidden state size.**

x_t is a vector in $d \times 1$

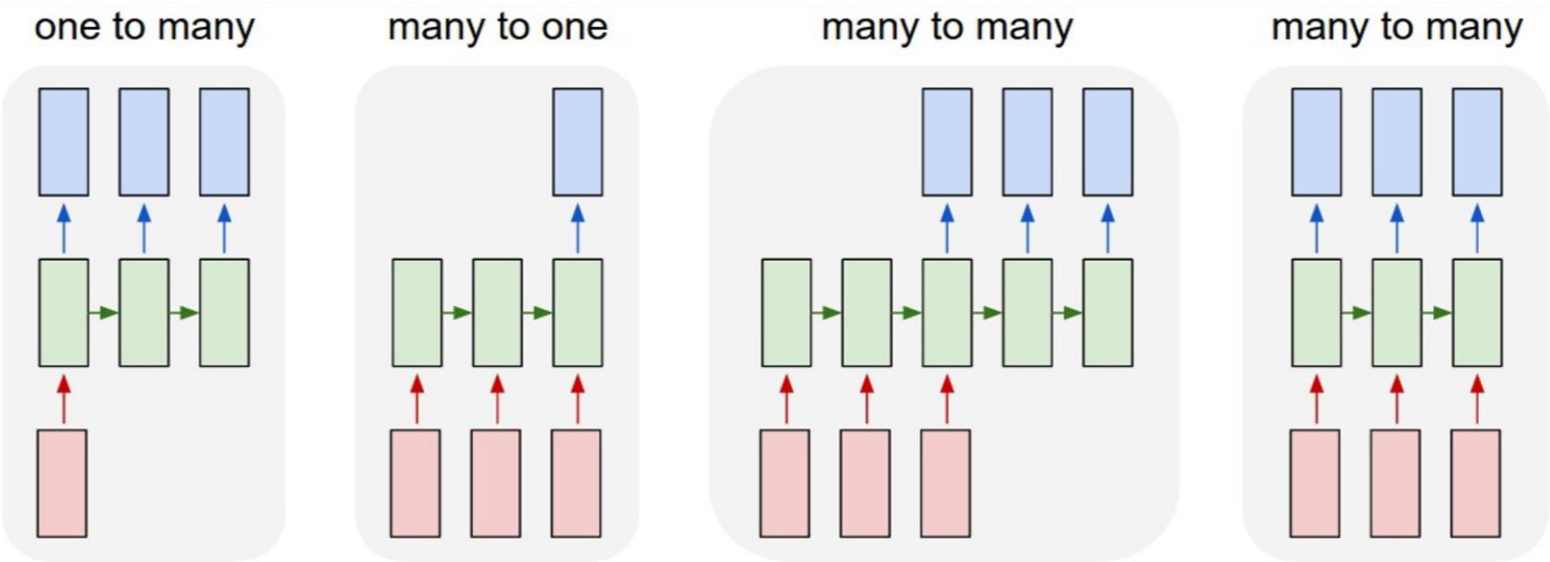
W_{xh} is a matrix in $R^{\{D_h \times d\}}$

W_{hh} is a matrix in $R^{\{D_h \times D_h\}}$

W_y is a matrix in $R^{\{|V| \times D_h\}}$

- Total parameter count is strictly bounded, completely independent of sequence length T.

RNN Design Patterns



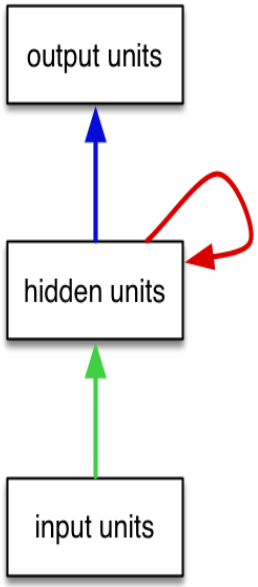
- One-to-Many: Image Captioning (Image -> Text sequence).
  - input image, output: [A man sitting on a chair]
- Many-to-One: Sentiment Analysis (Sequence -> Single label).
  - input “The food experience was fantastic“, output: [positive]
- Many-to-Many (Aligned): e.g., Part-of-Speech (POS) Tagging
  - input "The black cat jumped“, output: [Article, Adjective, Noun, Verb]
- Many-to-Many (Seq2Seq): Machine Translation (Encoder-Decoder framework).
  - input "I love cats“ [length=3], output: “J'aime les chats” [length=4]

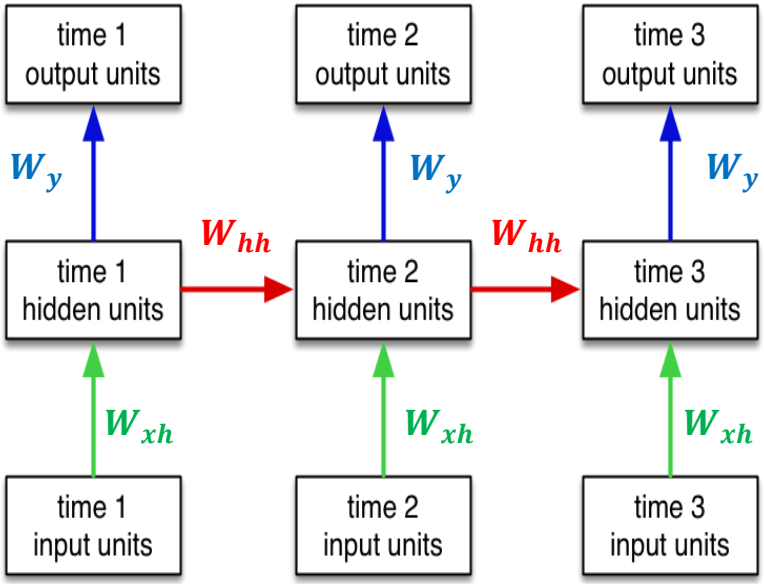
Unrolling the RNN

- To conceptualize how an RNN processes a sequence, we 'unroll' it in time.
- An unrolled RNN resembles a standard deep feed-forward network, where the 'depth' is the sequence length T.
- Nodes represent time steps, and edges represent the flow of data via W.
- This visualization is critical to understand backpropagation.
- The weights and biases are shared between all time steps

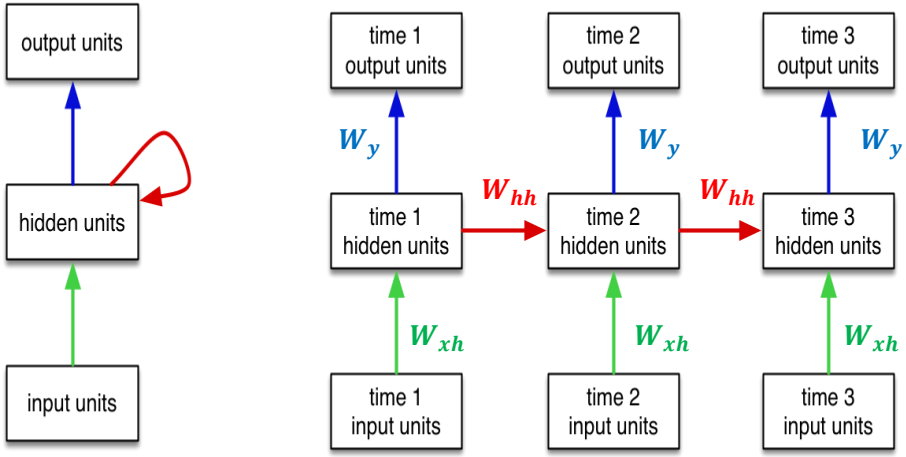
$$\hat{y}_t = f_{w_y}(h_t)$$

$$h_t = f_w(h_{t-1}, x_t)$$





Forward Pass Dynamics



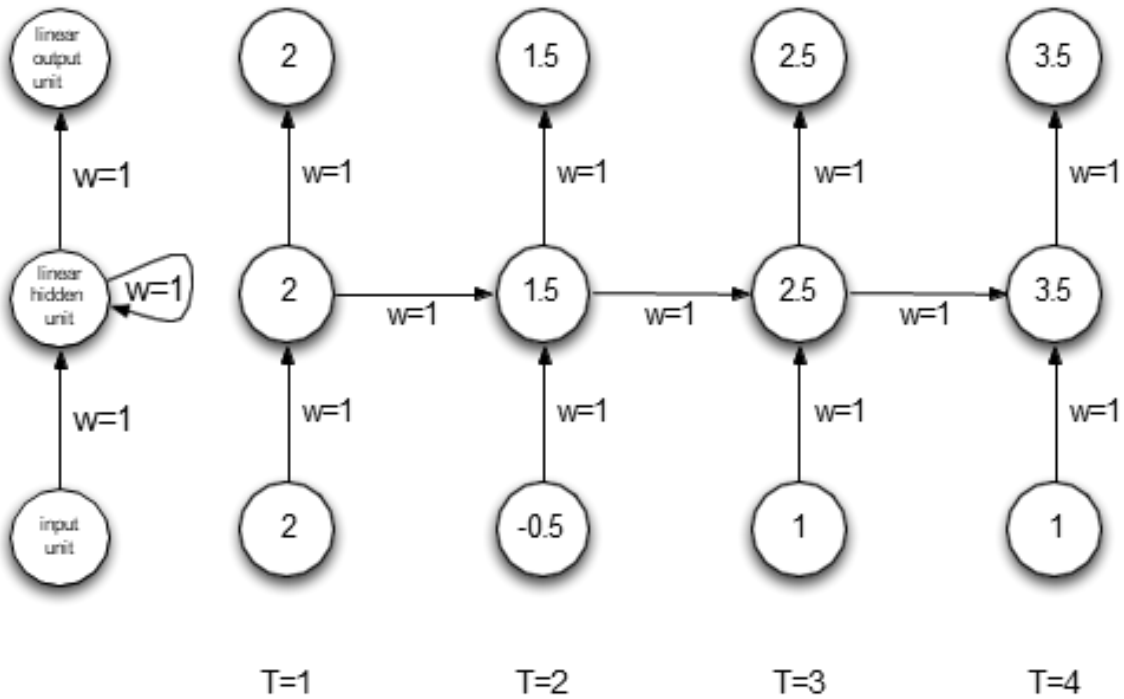
- **Let's trace the forward pass step-by-step.**
- Initialize h_0(usually as a vector of zeros).
- **At $t = 1$: $h_1 = \tanh(W_{hh} * h_0 + W_{xh} * x_1)$**
- **At $t = 2$: $h_2 = \tanh(W_{hh} * h_1 + W_{xh} * x_2)$**
- **And so on…**

Ignore bias b

Notice the composition of functions: h_2 contains information embedded from x_2 and x_1(via h_1)

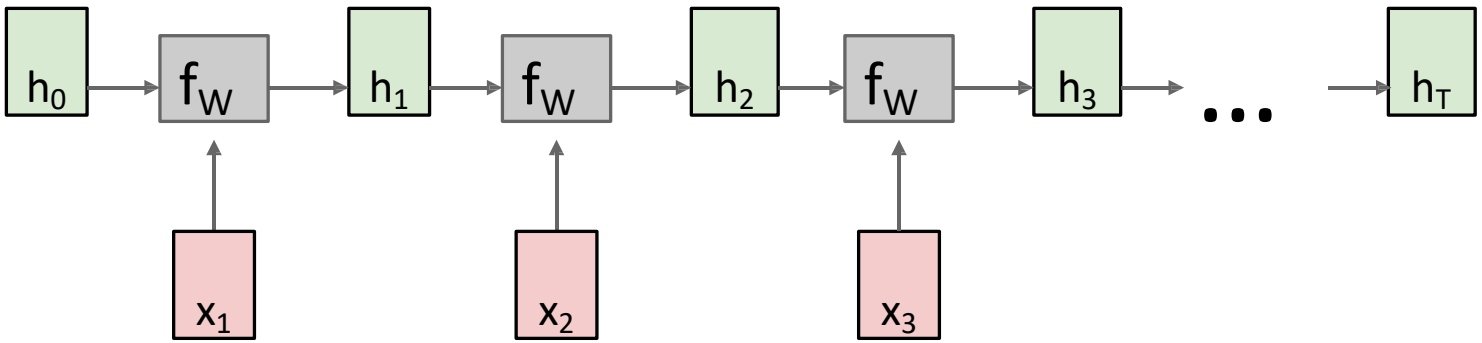
Simple RNN Example

- This one sums its inputs:



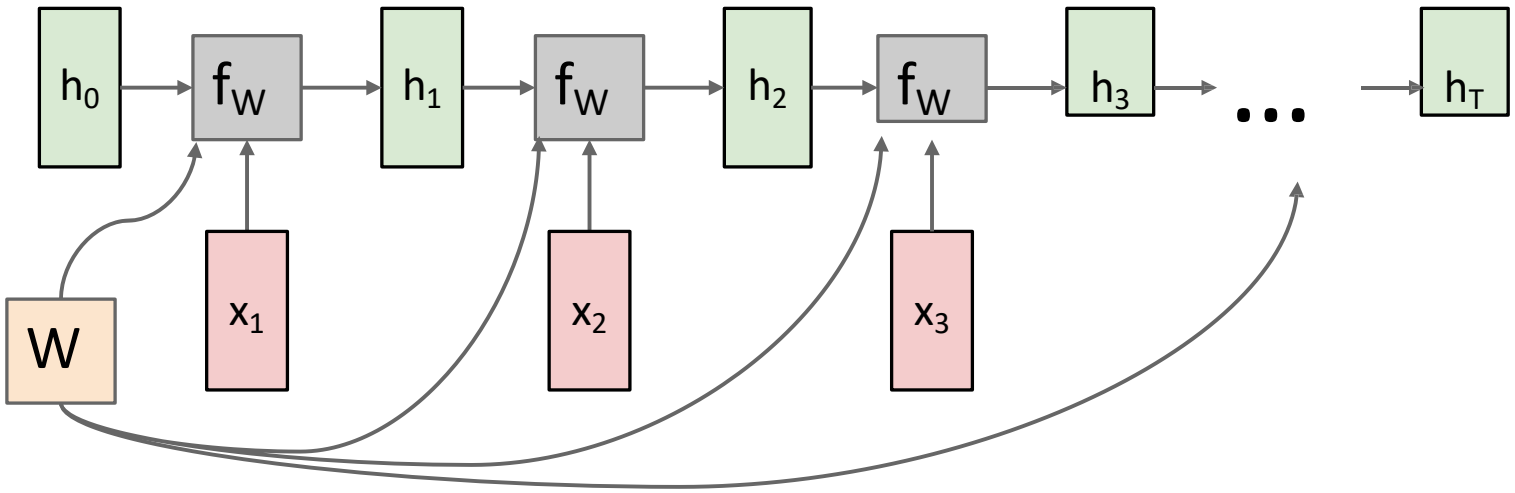
assume the **activation function to be the identity function** (meaning no non-linearity is applied) and the **biases are zero**

RNN: Computational Graph

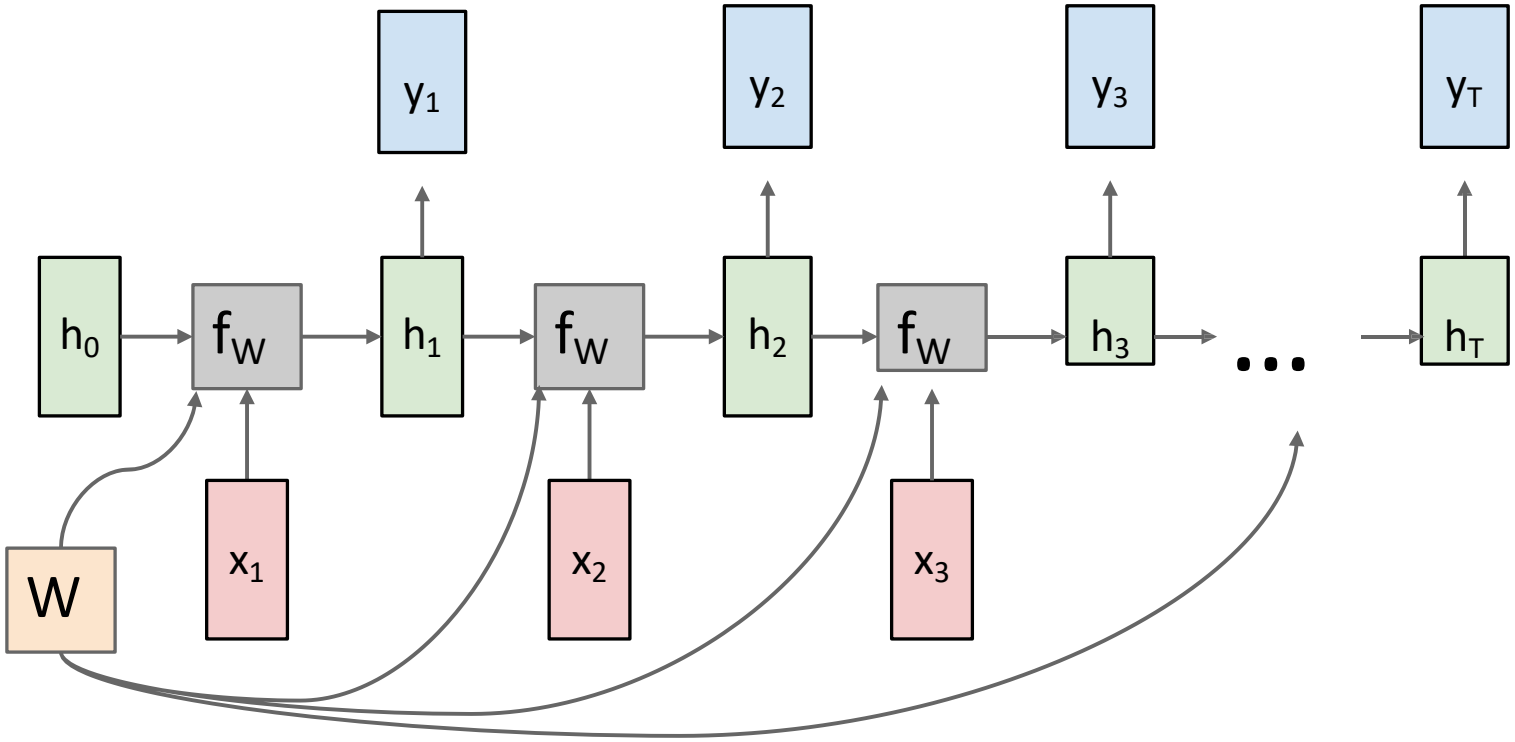


RNN: Computational Graph

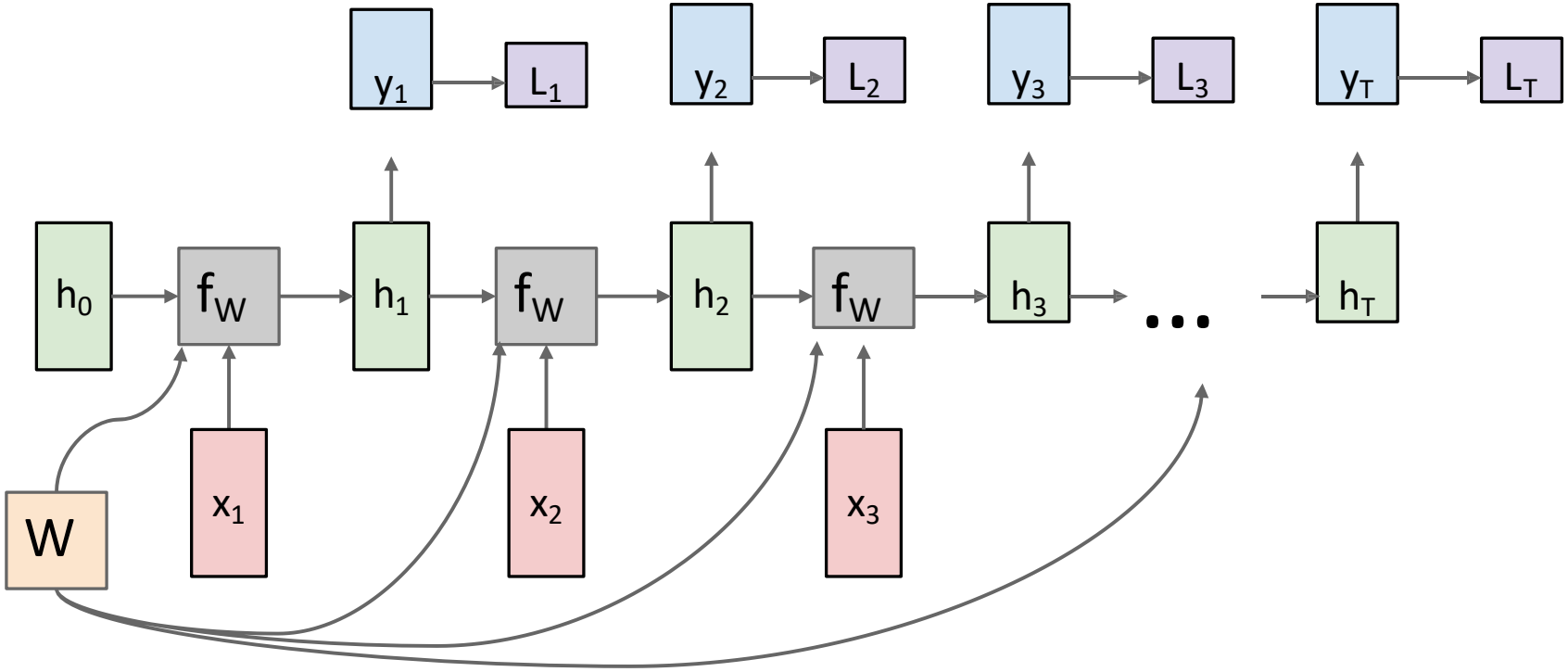
Re-use the same weight matrix at every time-step



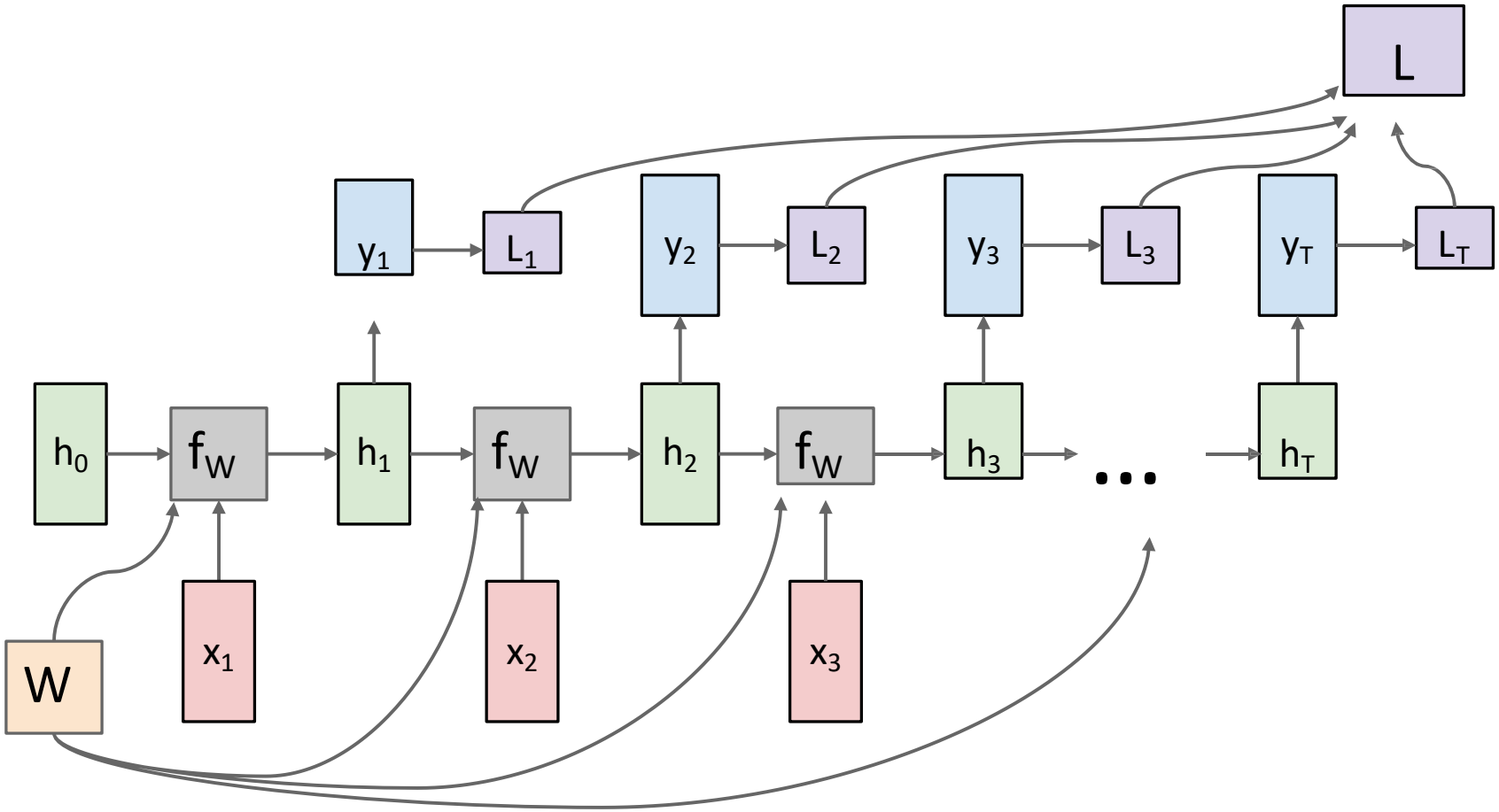
RNN: Computational Graph: Many to Many



RNN: Computational Graph: Many to Many



RNN: Computational Graph: Many to Many

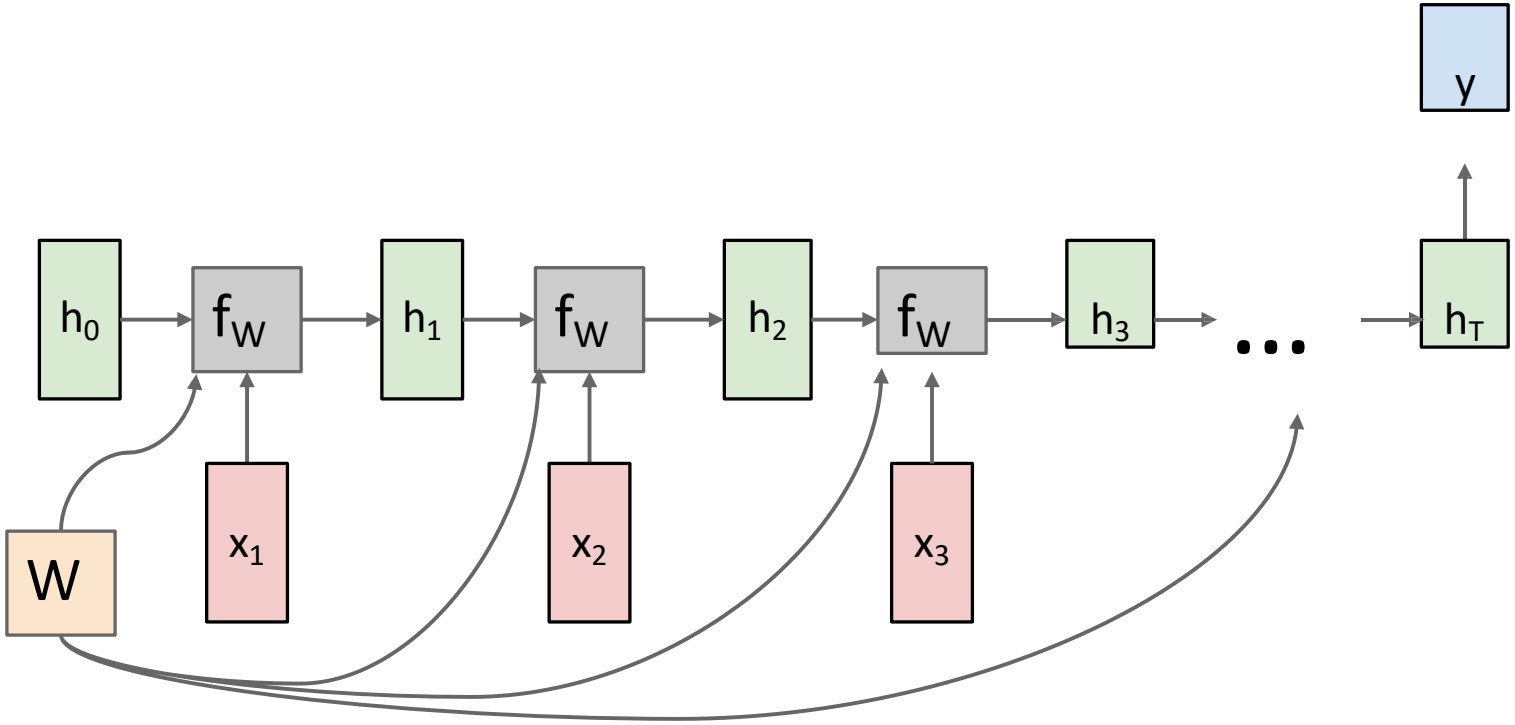


$$J_t(\theta) = -\sum_{j=1}^{|V|} y_{t,j} \log(\hat{y}_{t,j})$$

$$J(\theta) = \frac{1}{T} \sum_{t=1}^{T} J_t(\theta)$$

RNN: Computational Graph: Many to One

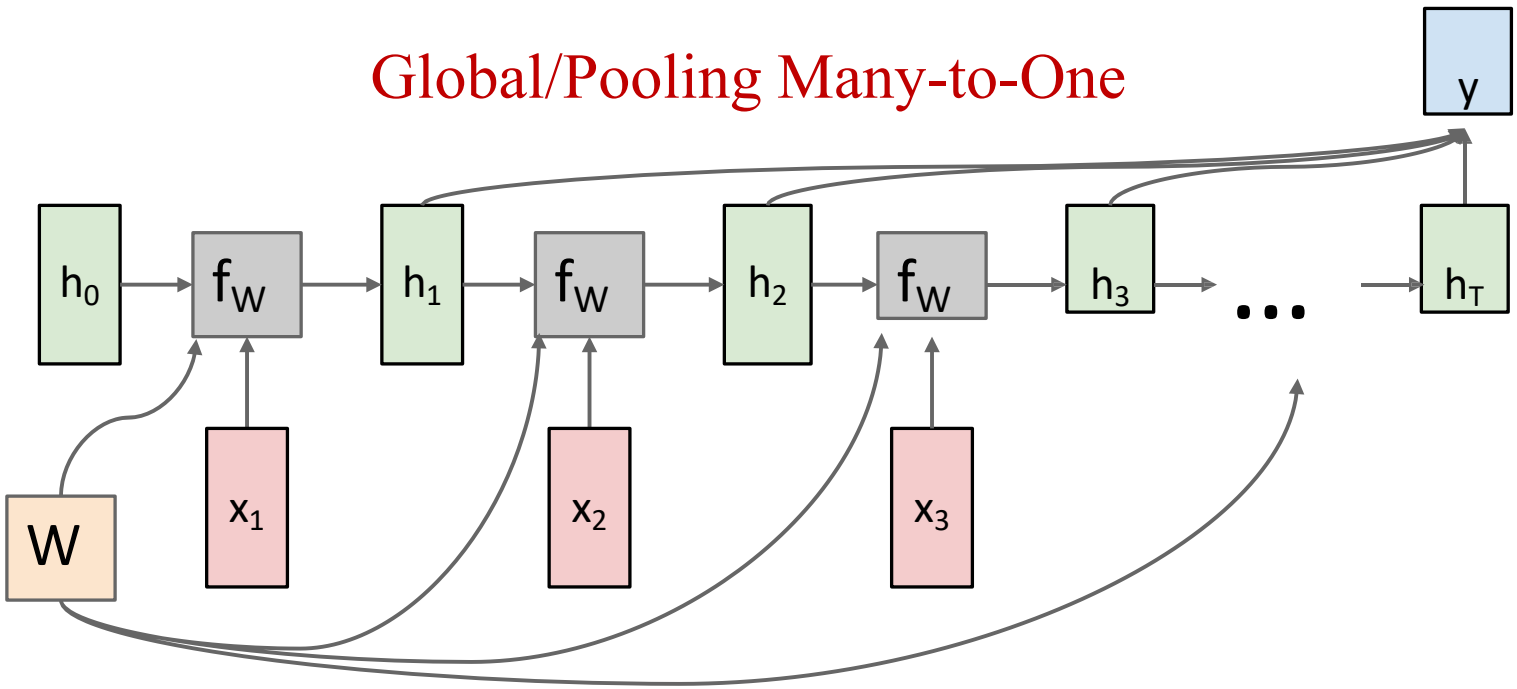
Standard Many-to-One



$$J(\theta) = -\sum_{j=1}^{|V|} y_j \log(\hat{y}_j) \text{ where } \hat{y}_j = softmax(W_y * h_T + b_y)$$

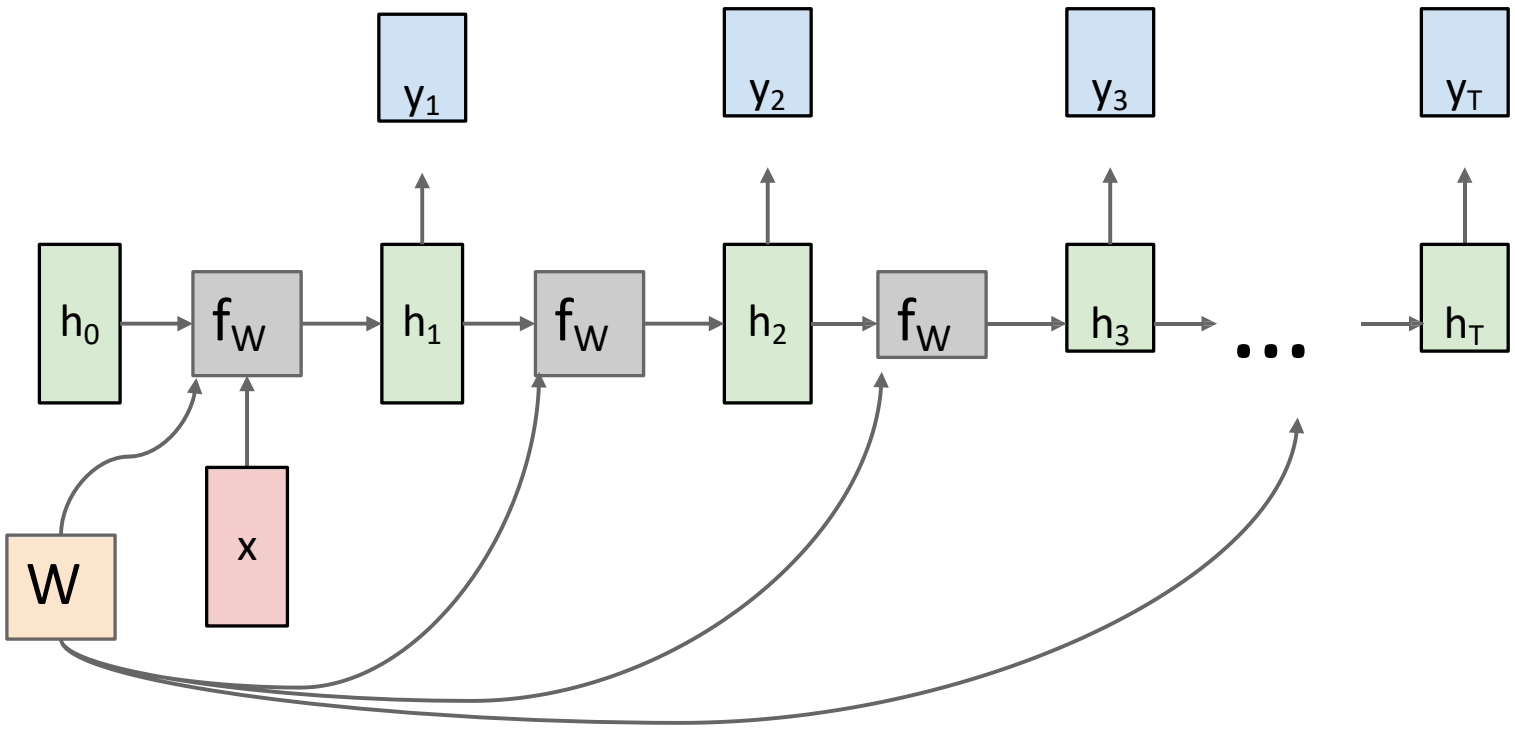
RNN: Computational Graph: Many to One

<u>Practical Example: Sentiment Analysis</u>:
Imagine the sentence: "The movie was not actually that good."
In a standard model, if h_T focuses only on the word "good," it might predict Positive.
In this model, the direct arrow from h_4 (representing the word "not") helps the final output y capture the negation immediately, leading to a correct Negative prediction

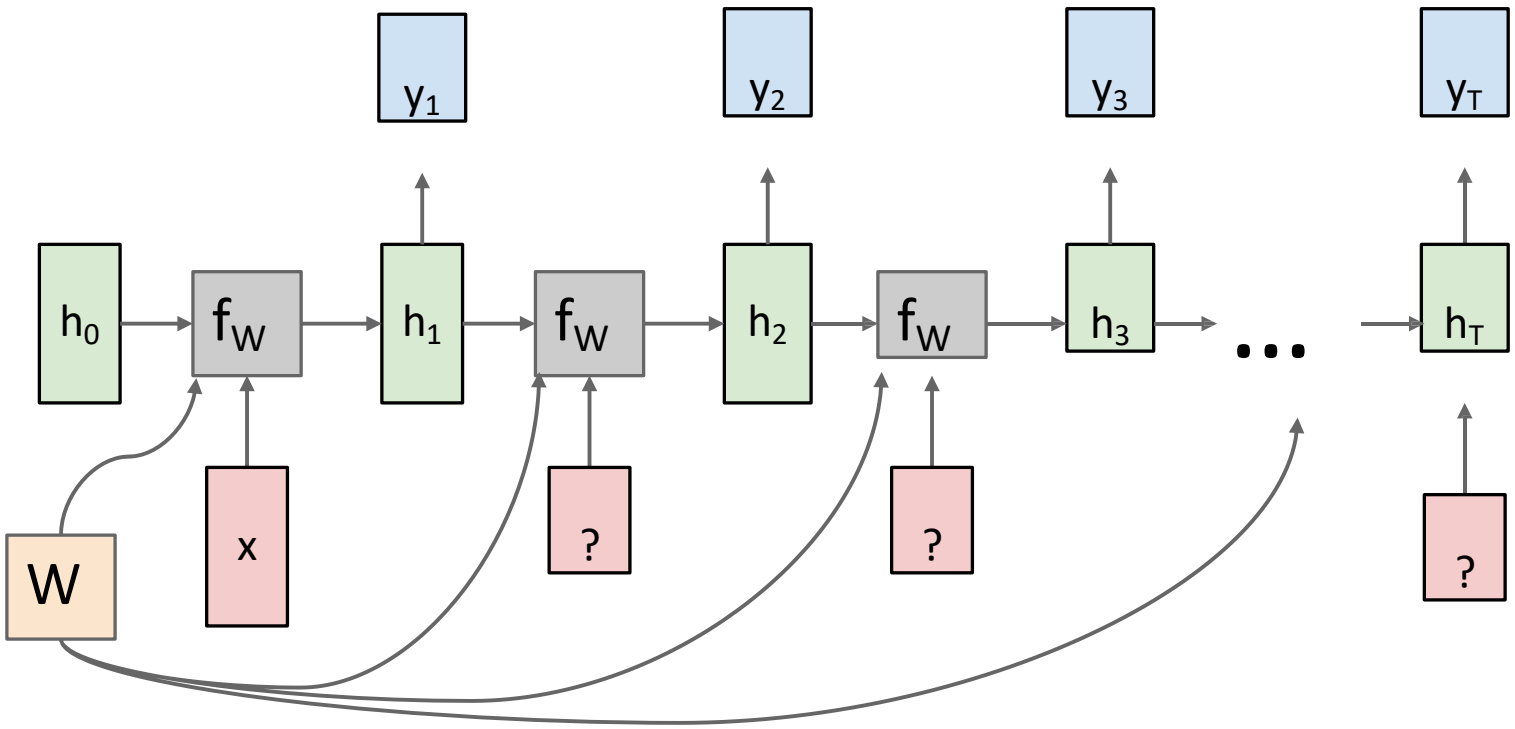


$$J(\theta) = -\sum_{j=1}^{|V|} y_j \log(\hat{y}_j) \text{ where } \hat{y}_t = softmax(W_y * \text{Aggregate}(h_1 \ldots h_T) + b_y)$$

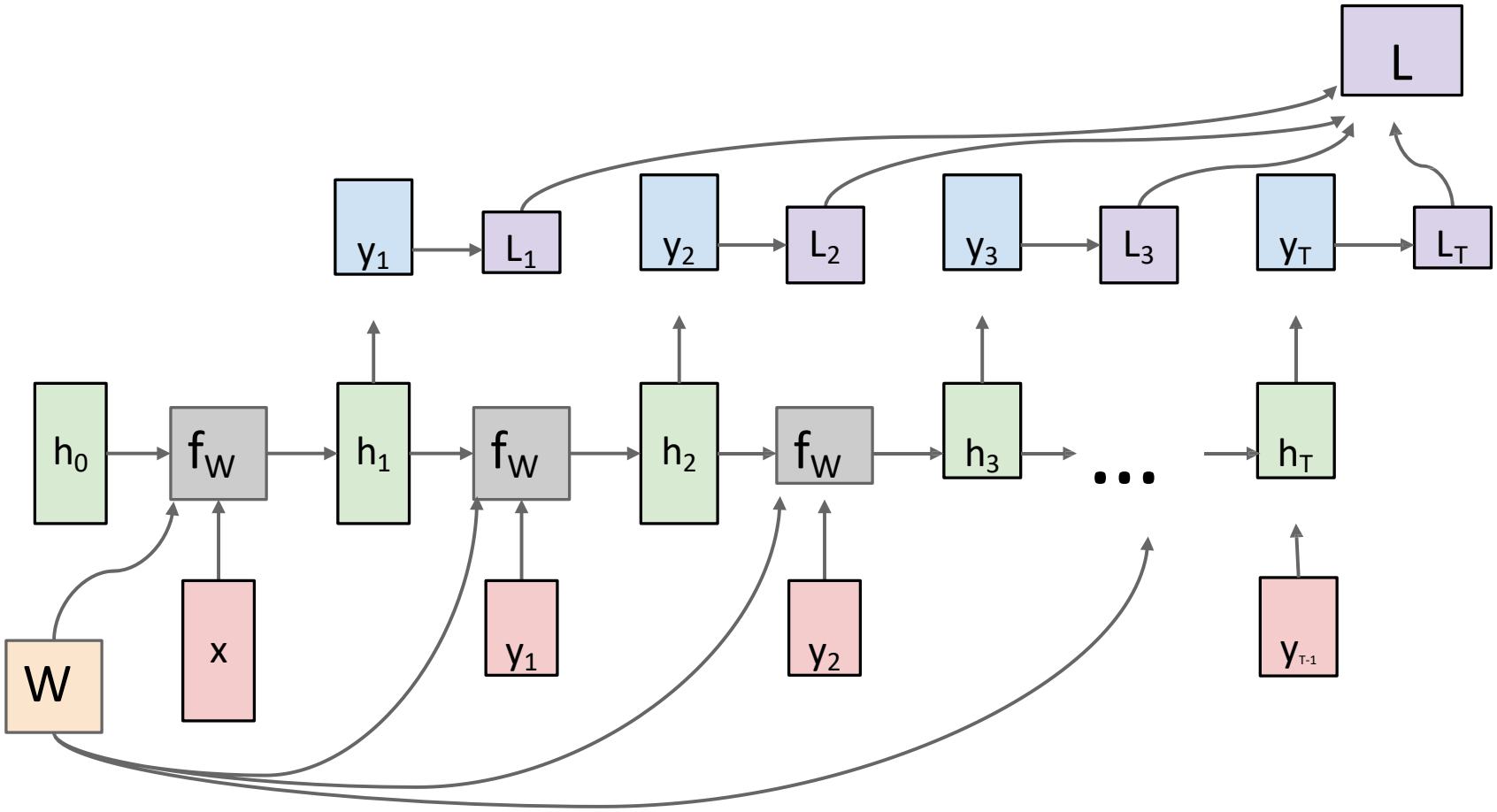
RNN: Computational Graph: One to Many



RNN: Computational Graph: One to Many



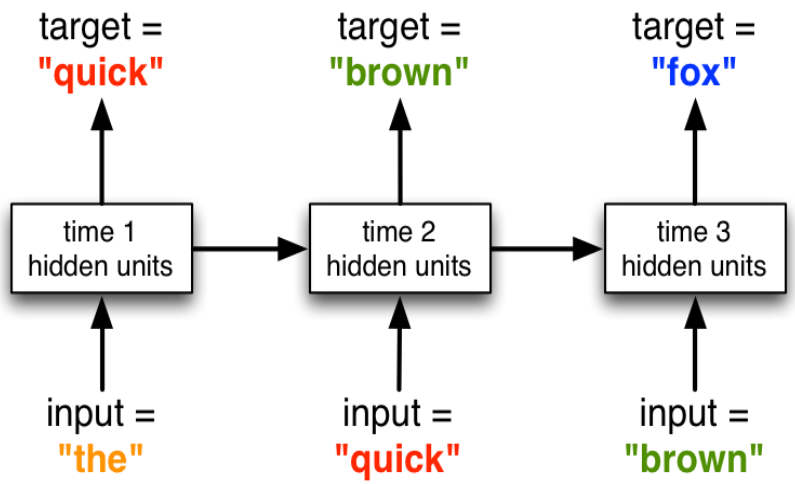
RNN: Computational Graph: One to Many



$$J_t(\theta) = -\sum_{j=1}^{|V|} y_{t,j} \log(\hat{y}_{t,j})$$

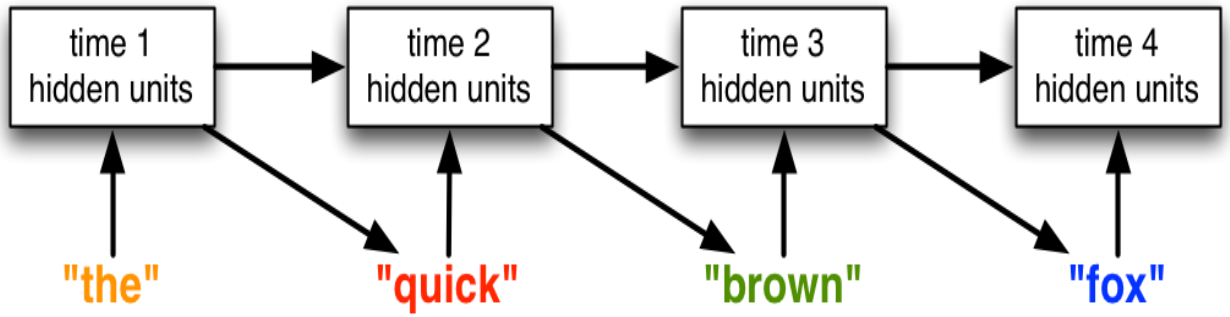
$$J(\theta) = \frac{1}{T} \sum_{t=1}^{T} J_t(\theta)$$

Language Modeling



- This is one way to use RNNs as a language model
- Each word is represented as an embedding vector, the model predicts a distribution, and we can train it with cross-entropy loss.
- During the **Training Phase** of a Language Model, we use a **Many-to-Many** architecture.
  - We give the model all the correct words as inputs (Teacher Forcing) and ask it to predict the next ones.
- This model can learn long-distance dependencies.

Language Modeling-Generation



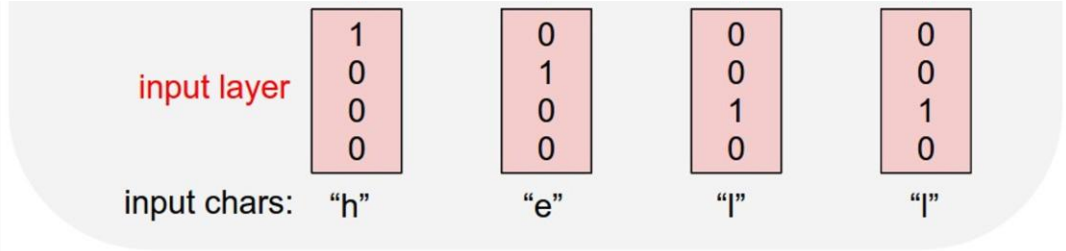
- When we generate from the model (i.e. compute samples from its distribution over sentences), the outputs feed back into the network as inputs.
- This is **One-to-Many**. We give it one word, and it must use its own previous predictions as the next inputs to "dream" up a sentence.

Character-level Language Model

A more practical example: Character-level **Language Model**

Vocabulary: [h,e,l,o]

Example training sequence: “hello”

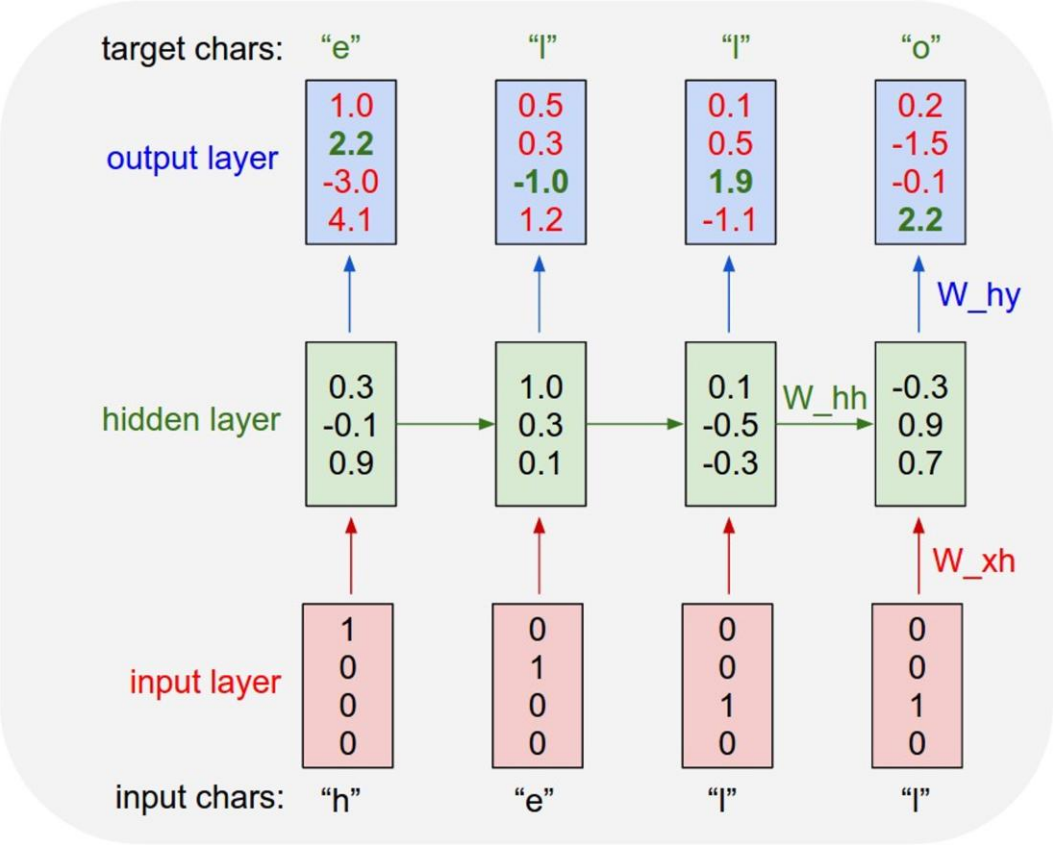


$$h_t = \tanh(W_{hh} h_{t-1} + W_{xh} x_t)$$

A more practical example: Character-level **Language Model**

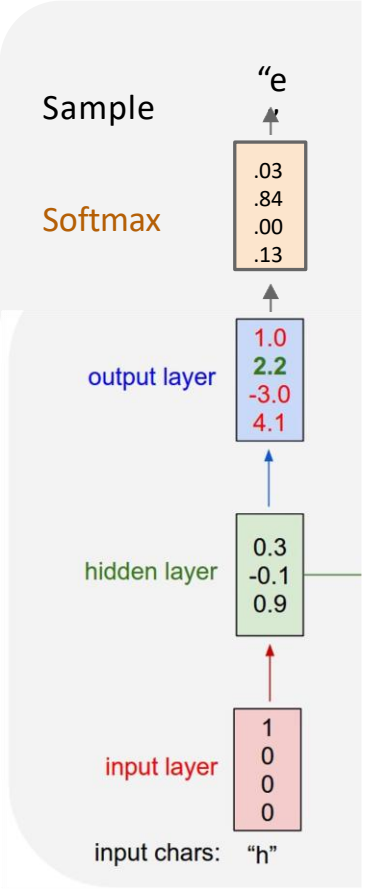
Vocabulary: [h,e,l,o]

Example training sequence: “hello”



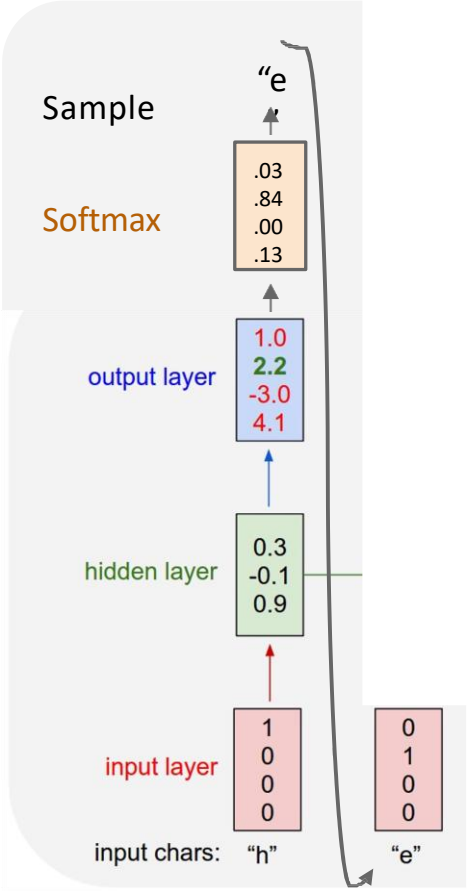
Vocabulary:
[h,e,l,o]

At test-time sample characters one at a time, feed back to model



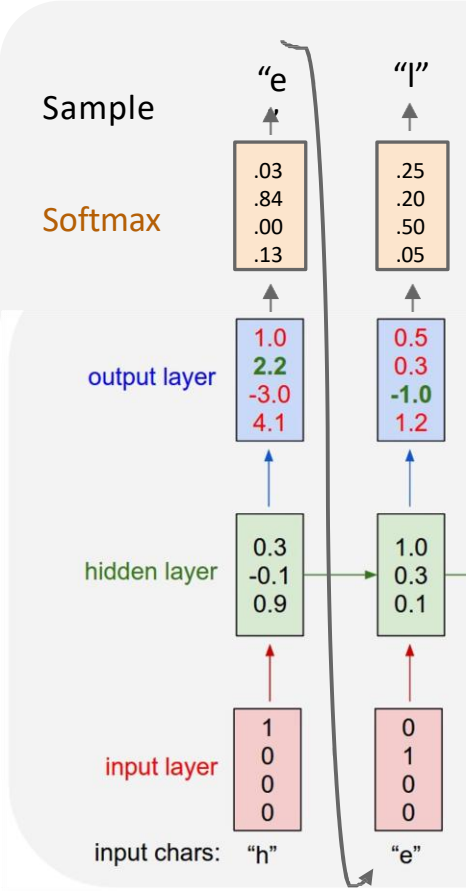
Vocabulary:
[h,e,l,o]

At test-time sample characters one at a time, feed back to model



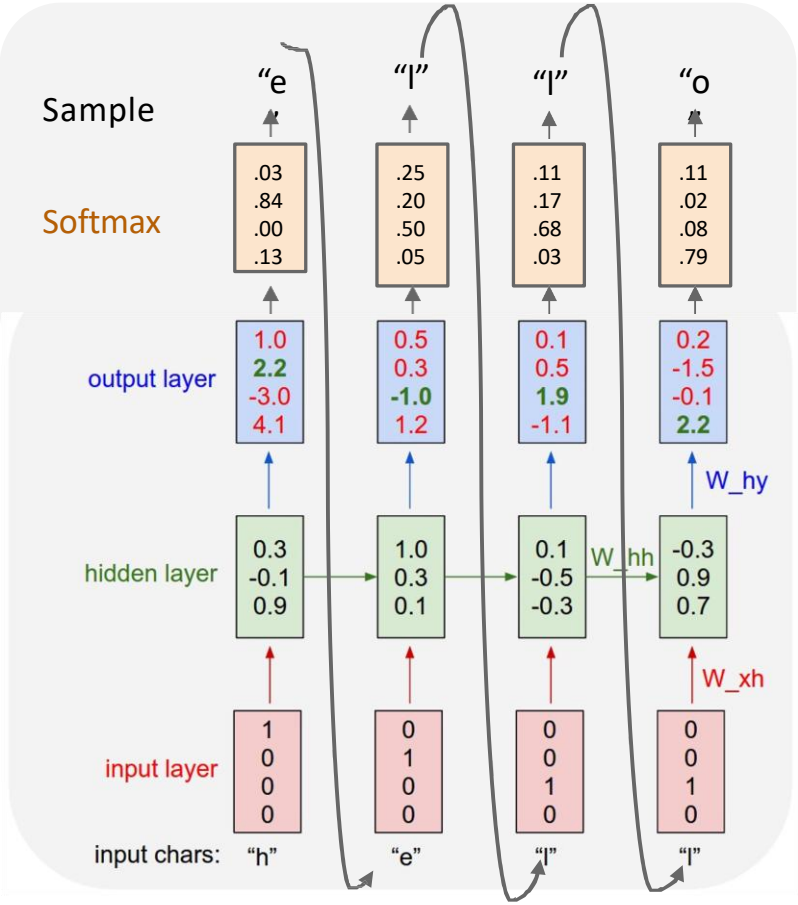
Vocabulary:
[h,e,l,o]

At test-time sample characters one at a time, feed back to model



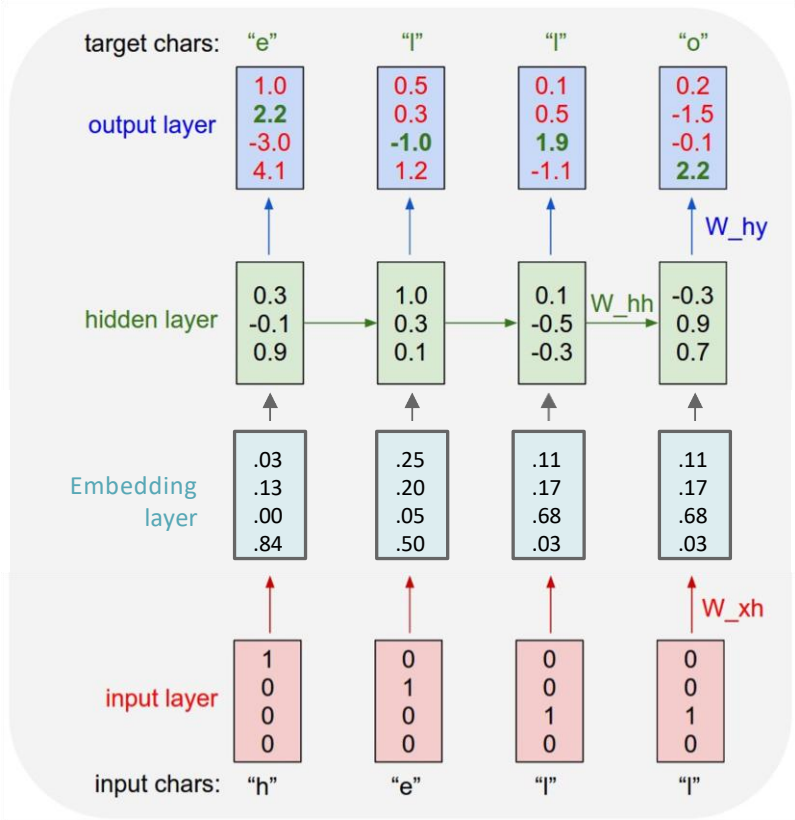
Vocabulary:
[h,e,l,o]

At test-time sample characters one at a time, feed back to model



$$
\begin{bmatrix} w_{11} & w_{12} & w_{13} & w_{14} \\ w_{21} & w_{22} & w_{23} & w_{14} \\ w_{31} & w_{32} & w_{33} & w_{14} \\ w_{41} & w_{42} & w_{43} & w_{44} \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} w_{11} \\ w_{21} \\ w_{31} \\ w_{41} \end{bmatrix}
$$

Matrix multiplication with a one-hot vector just extracts a column from the weight matrix. We often put a separate embedding layer between the input and hidden layers.



From Geoff Hinton's course, an example of a paragraph generated by an RNN language model one character at a time:

> He was elected President during the Revolutionary War and forgave Opus Paul at Rome. The regime of his crew of England, is now Arab women's icons in and the demons that use something between the characters' sisters in lower coil trains were always operated on the line of the ephemerable street, respectively, the graphic or other facility for deformation of a given proportion of large segments at RTUS). The B every chord was a "strongly cold internal palette pour even the white blade."

J. Martens and I. Sutskever, 2011. Learning recurrent neural networks with Hessian-free optimization.

http://machinelearning.wustl.edu/mlpapers/paper_files/ICML2011Martens_532.pdf

From example to train an RNN to write like William Shakespeare, an example of a paragraph generated by an RNN language model one character at a time:

at first:

```
tyntd-iafhatawiaoihrdemot  lytdws  e ,tfti, astai f ogoh eoase rrranbyne 'nhthnee e
plia tklrgd t o idoe ns,smtt   h ne etie h,hregtrs nigtike,aoaenns lng
```

train more

```
"Tmont thithey" fomesscerliund
Keushey. Thom here
sheulke, anmerenith ol sivh I lalterthend Bleipile shuwy fil on aseterlome
coaniogennc Phe lism thond hon at. MeiDimorotion in ther thize."
```

train more

```
Aftair fall unsuch that the hall for Prince Velzonski's that me of
her hearly, and behs to so arwage fiving were to it beloge, pavu say falling misfort
how, and Gogition is so overelical and ofter.
```

train more

```
"Why do what that day," replied Natasha, and wishing to himself the fact the
princess, Princess Mary was easier, fed in had oftened him.
Pierre aking his soul came to the packs and drove up his father-in-law women.
```

Training RNN

Training an RNN: Sequence Cross-Entropy

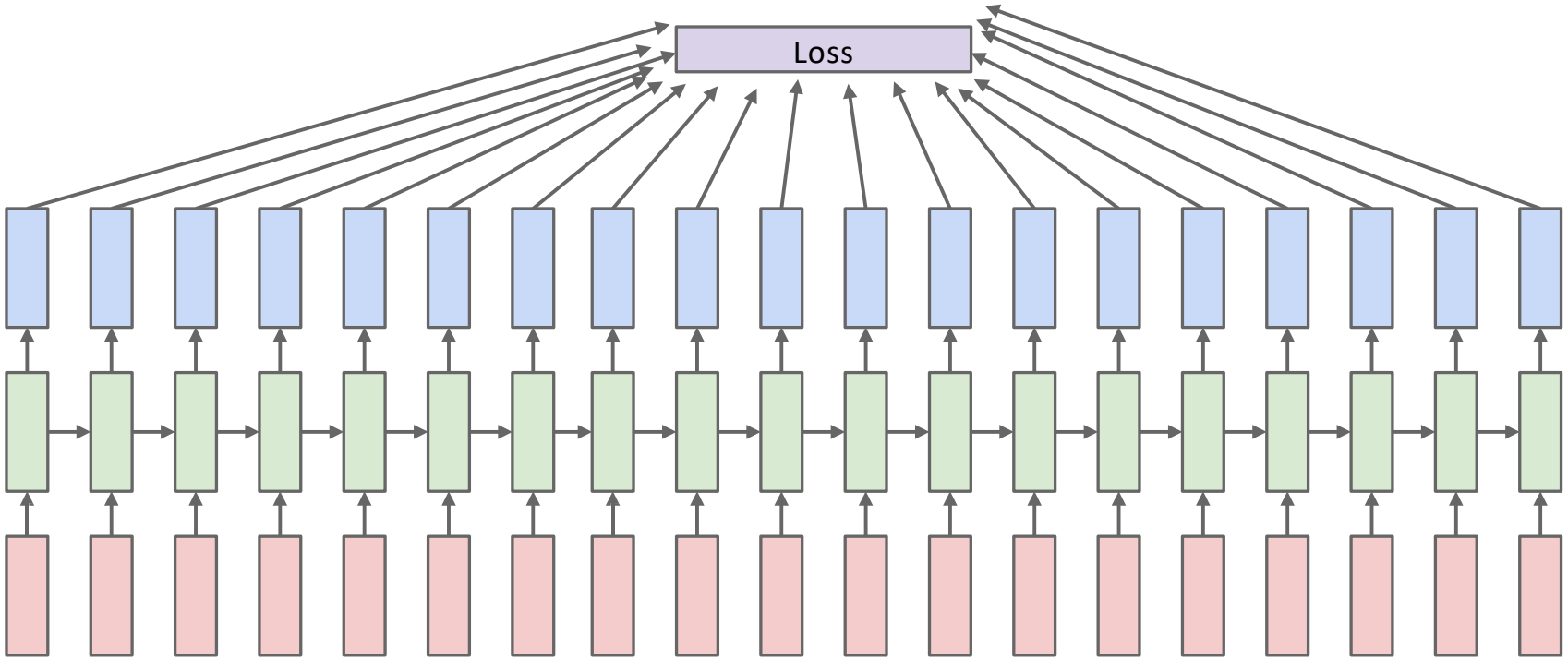
- At each time step t, the model outputs a probability distribution $\hat{\boldsymbol{y}}_{\boldsymbol{t}}$.
- The true target y_t is a one-hot vector representing the actual next word.
- We compute the Cross-Entropy Loss at each time step:

$$J_t(\theta) = -\sum_{j=1}^{|V|} y_{t,j} \log(\hat{y}_{t,j})$$

- The total Cross-Entropy Loss calculation changes depending on the architecture.
- To train the network, we minimize $J(\theta)$by updating weight matrices W_{hh}, W_{xh}, W_y

Backpropagation Through Time (BPTT)

- Because parameters are shared across time, a loss at time t depends on the hidden state at time t, which depends on t-1, and so on.
- We must backpropagate the error signal backwards through the unrolled temporal graph.
- This specific application of backpropagation is called Backpropagation Through Time (BPTT).



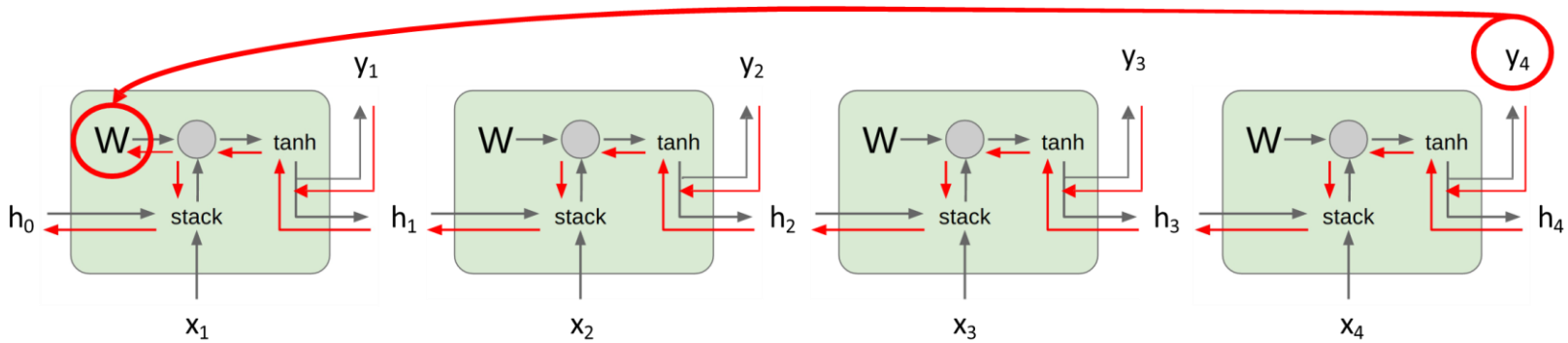
BPTT: The Multivariate Chain Rule

- If a variable depends on multiple intermediate paths, its derivative is the sum of the derivatives along all paths.
- In an RNN, the matrix W_{hh} is used at step 1, step 2, ..., step t.
- To find the effect of W_{hh} on the loss at time t, we must sum its contributions from EVERY preceding time step j.
- Since $j = \sum j_t$, the total derivative w.r.t W_{hh} is:

$$\frac{\partial J}{\partial W_{hh}} = \sum_{t=1}^{T} \frac{\partial J_t}{\partial W_{hh}}$$

- Let's isolate a single timestep loss, J_t, and see how it affects W_{hh}.

BPTT: (Chain Rule Expansion)



Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks",ICML 2013

- $J_t$depends on h_t. But h_t depends on W_{hh}AND h_{t-1}
- h_{t-1} depends on W_{hh}AND h_{t-2} and so on.
- Applying the multivariate chain rule:

$$\frac{\partial J_t}{\partial W_{hh}} = \sum_{k=1}^{t} \frac{\partial J_t}{\partial h_t} \frac{\partial h_t}{\partial h_k} \frac{\partial h_k}{\partial W_{hh}}$$

$\sum_{k=1}^{t}$ This represents summing the contributions from every preceding time step k up to the current step t

$\frac{\partial h_t}{\partial h_k}$ This term accounts for the "hidden-to-hidden" chain of dependencies through time

The Critical Term: The Jacobian

- In the BPTT equation, the most important term is the Jacobian matrix: $\frac{\partial h_t}{\partial h_k}$.
- This matrix (size $D_h \times D_h$) measures how a small change in the hidden state at past time k affects the state at future time t.
- This term dictates whether the network can remember long-term context.

Why the Jacobian Matters

- If $\frac{\partial h_t}{\partial h_k}$ is very small (approaches 0), the loss J_t provides no gradient signal to update weights for time k
- The network suffers from 'amnesia'
- If $\frac{\partial h_t}{\partial h_k}$ is very large, the gradient signal explodes

Expanding the Jacobian

- Because state transitions are sequential, this derivative is expanded via the chain rule as a product of consecutive Jacobians:

$$\frac{\partial h_t}{\partial h_k} = \prod_{j=k+1}^{t} \frac{\partial h_j}{\partial h_{j-1}}$$

The Local Derivative (dh_j / dh_j-1)

- Recall $h_j = \tanh\left(W_{hh} h_{j-1} + W_{xh}\, x_j + b\right)$
- Taking the derivative w.r.t h_{j-1}:

$$\frac{\partial h_j}{\partial h_{j-1}} = [diag\left(\, f'(z_j)\right)] * \; W_{hh}$$

Where f' is the derivative of tanh, and $diag()$ turns the vector into a diagonal matrix.

The Tanh Derivative

- The derivative of $\tanh(x)$ is: $1 - \tanh^2(x)$

$$\frac{\partial h_j}{\partial h_{j-1}} = [\mathrm{diag}\left(1 - \tanh^2\left(W_{hh}h_{j-1} + W_{hx}x_j + b\right)\right)] * W_{hh}$$

- The maximum value of this derivative is exactly 1 (when $\boldsymbol{x} = \mathbf{0}$).
- For all other values of x, the derivative is strictly less than 1.
- Keep this bounding characteristic in mind.

Repeated Multiplication

- Substituting back into our product series:

$$\frac{\partial h_t}{\partial h_k} = \prod_{j=k+1}^{t} [diag\left(f'(z_j)\right)] * W_{hh}$$

- We are multiplying the EXACT SAME weight matrix W_{hh} repeatedly ($t - k$ times).
- What happens when you multiply a matrix by itself N times?

Spectral Radius Theory

- The behavior of $\|W_{hh}\|^{\boldsymbol{t-k}}$ is governed by the eigenvalues of W_{hh}
- Let λ_{max} be the largest eigenvalue (spectral radius)
- Theorem (Pascanu et al., 2013):
  - If λ_{max} < 1, is sufficient for gradients exponentially vanish.
  - If λ_{max} > 1, is necessary for gradients exponentially explode.

The Exploding Gradient

- If λ_{max} > 1, the product $(\lambda_{max})^{t-k}$ grows exponentially to infinity.
- When gradients explode and the optimization surface contains steep cliffs –> Model becomes unstable and "crashes".

Numerical Example: Exploding Gradients

- Model a simplified scalar RNN over $\boldsymbol{T} = \mathbf{50}$ steps.
- Assume the effective weight $||\boldsymbol{W}|| = \mathbf{1.5}$
- Recall:

$$\frac{\partial J_t}{\partial W_{hh}} = \sum_{k=1}^{t} \frac{\partial J_t}{\partial h_t} \frac{\partial h_t}{\partial h_k} \frac{\partial h_k}{\partial W_{hh}}$$

$$\left\| \frac{\partial h_t}{\partial h_k} \right\| \leq \prod_{j=k+1}^{t} \left\| diag\left(f'(z_j) \right) \right\| * \|W_{hh}\|$$

- The bound of $\left\| \frac{\partial h_t}{\partial h_k} \right\|$ is entirely driven by: $\|W_{hh}\|^{\boldsymbol{t-k}}$
- Gradient scaling over 50 steps: $\|W_{hh}\|^{\boldsymbol{t-k}} = 1.5^{50} \cong 637{,}621{,}500.$
- Recall:

$$\text{New Weight} = Old\ Weight - (Learning\ Rate\ \times\ \text{Gradient})$$

- If your gradient is 637 million and your learning rate is 0.01, your weight change is multiplied by 6.37 million.
- This massive jump moves the weights so far away from their optimal values that the model becomes mathematically "broken“ → Weights are instantly destroyed.

Symptoms of Exploding Gradients

- Model weights become NaN (Not a Number) due to floating-point overflow.
- Loss immediately diverges to infinity.
- Training process crashes abruptly.

The Vanishing Gradient

- If λ_{max} < 1, the product $(\lambda_{max})^{t-k}$ shrinks exponentially to 0.
- Vanishing gradients are insidious. The model doesnot crash but it just fails to learn.
- Early layers (early time steps) receive effectively zero gradient update.

Numerical Example: Vanishing Gradients

- Model a simplified scalar RNN over $\boldsymbol{T} = \mathbf{50}$ steps.
- Assume the effective weight $||\boldsymbol{W}|| = 0.\mathbf{5}$
- Gradient scaling over 50 steps: $\|W_{hh}\|^{\boldsymbol{t-k}} = 0.5^{50} \cong$ 8.88 x 10^{-16}.
- Floating-point precision treats this as absolute zero.
- Step $T = 1$ receives absolutely no learning signal from the loss at step 50.

Symptoms of Vanishing Gradients

- The model performs well on short sequences but fails on length 20+.
- In text generation, the model produces locally fluent text but loses the overarching topic entirely.
- The RNN effectively acts as a short-window n-gram model.

Fixing Exploding Gradients: Gradient Clipping

- Exploding gradients are mathematically severe but easy to fix heuristically.
- Solution: Gradient Clipping (Pascanu et al., 2012).
- Algorithm: Compute the L2 norm of the gradient vector $||g||$
- If $||g|| > threshold$, scale it down:

$$g = \frac{g * (threshold)}{||g||}$$

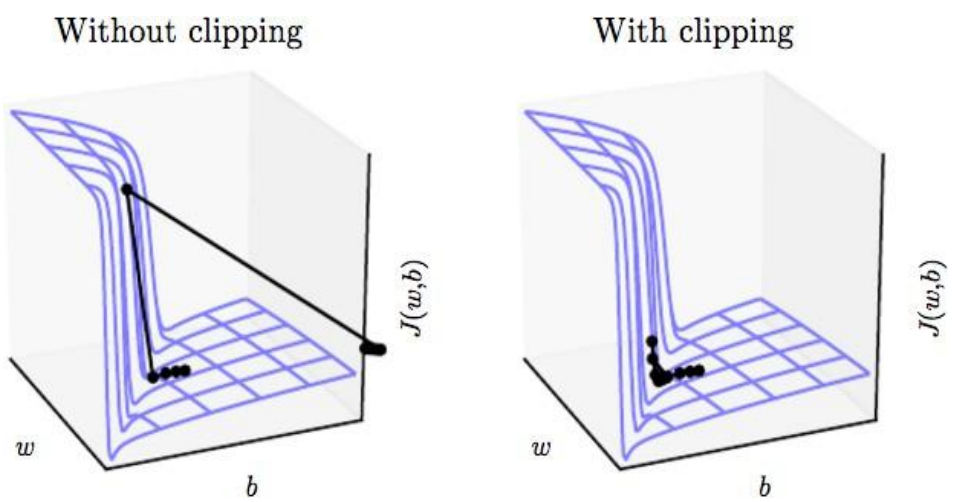
Why Clipping Works

- Clipping preserves the exact multidimensional direction of the gradient vector.
- It strictly bounds its maximum length (putting a 'speed limit' on the update).
- This prevents the model from stepping off an optimization cliff.

The Cliff: As the black dots (the training steps) move through the ravine, one step lands exactly on the face of an extremely steep cliff.

The Explosion: Because the slope is nearly vertical, the mathematical gradient becomes massive.

The Catapult: When the model multiplies its learning rate by that massive gradient, it results in a gigantic update. The next black dot is propelled completely out of the optimal valley, undoing all previous learning.



Pascanu et al. (2013)

Why Clipping Cannot Fix Vanishing Gradients

- You cannot 'clip' a vanished gradient.
- If the signal is 10^{-16}, it is indistinguishable from floating-point numerical noise.
- Scaling it up multiplies the noise, destroying the signal.
- Vanishing gradients require a fundamental architectural change.

LSTM

Long Short-Term Memory (LSTM)

- **Long Short-Term Memory** (LSTM) is a popular architecture that makes it easy to remember information over long time periods.
- Introduced by Hochreiter and Schmidhuber (1997).
- Designed specifically to solve the vanishing gradient problem.
- LSTMs separate memory into two tracks.
- It is composed of memory cells which have controllers saying when to store or forget information.

The Dual-State Architecture

- A Vanilla RNN has one state h_t.
- An LSTM separates the state into two distinct vectors:
  1. The Cell State (C_t): Long-term memory running as a structural backbone.
  2. The Hidden State (h_t): Short-term memory used for immediate predictions.

Vanilla RNN

$$h_t = \tanh(W_{hh} h_{t-1} + W_{xh} x_t)$$

$$\boxed{h_t = \tanh\left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right)}$$

LSTM

Four gates

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

Cell state

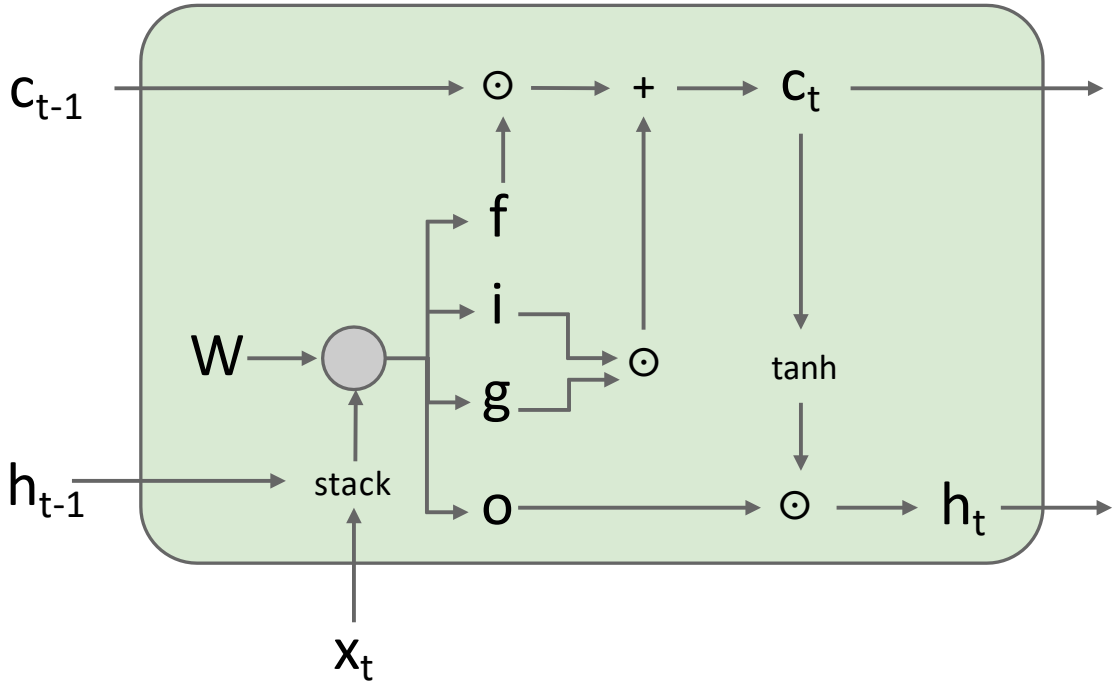
$$c_t = f \odot c_{t-1} + i \odot g$$

Hidden state

$$h_t = o \odot \tanh(c_t)$$

Long Short Term Memory (LSTM)

[Hochreiter et al., 1997]



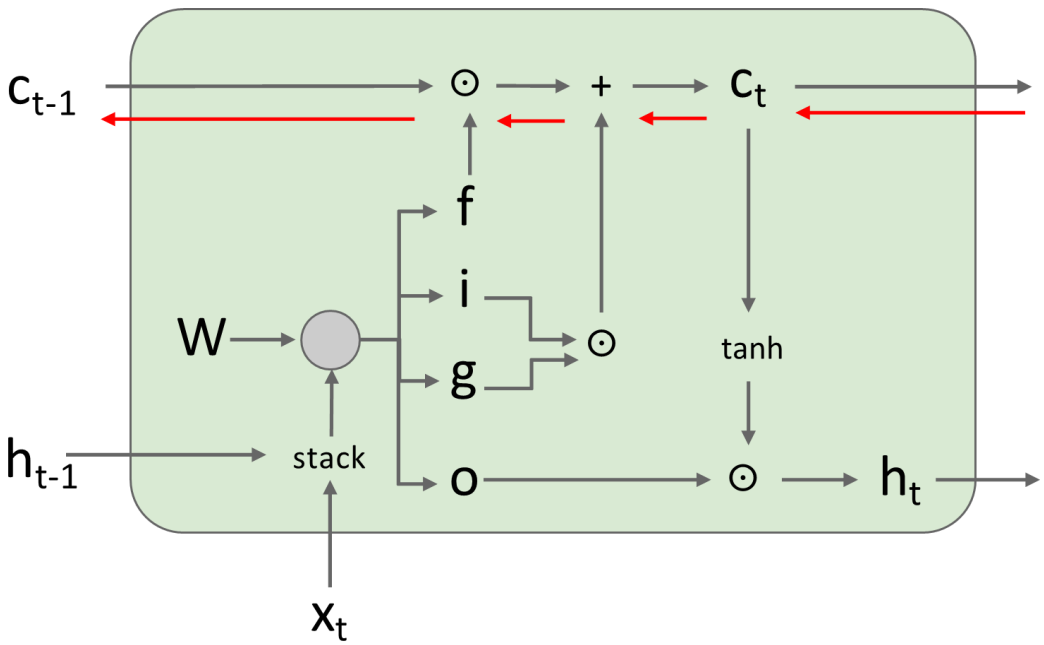
$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

The Cell State: The Conveyor Belt

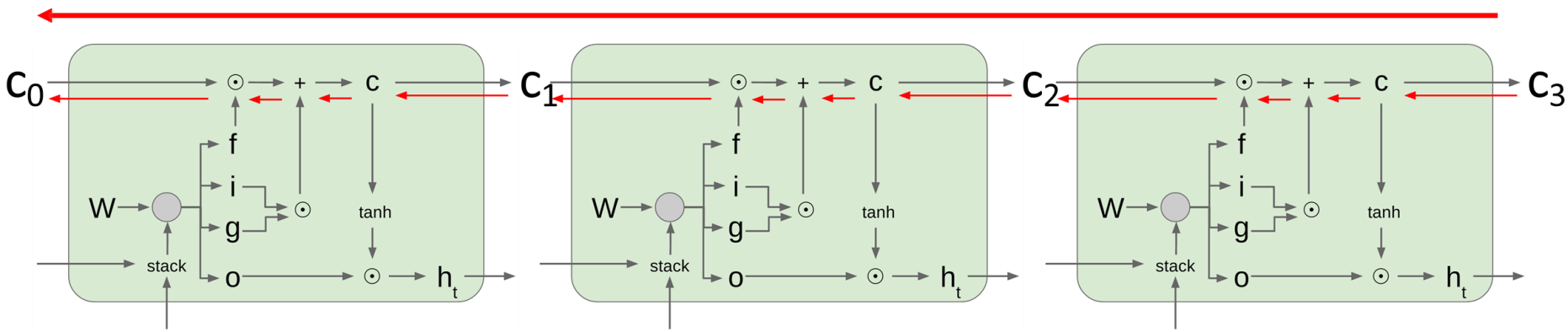
- The Cell State (C_t) acts as a high-speed data highway (conveyor belt) running straight down the entire sequence.
- It undergoes strictly linear interactions (pointwise addition and multiplication).
- It is very easy for gradients to flow backwards along it unchanged.



Backpropagation from c_t to c_{t-1} only elementwise multiplication by f, no matrix multiply

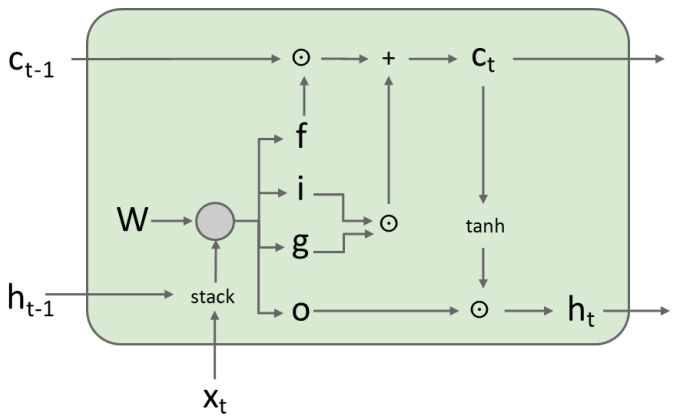
LSTM Gradient Flow

Uninterrupted gradient flow!



More on this in the next few slides

Regulating Information with Gates



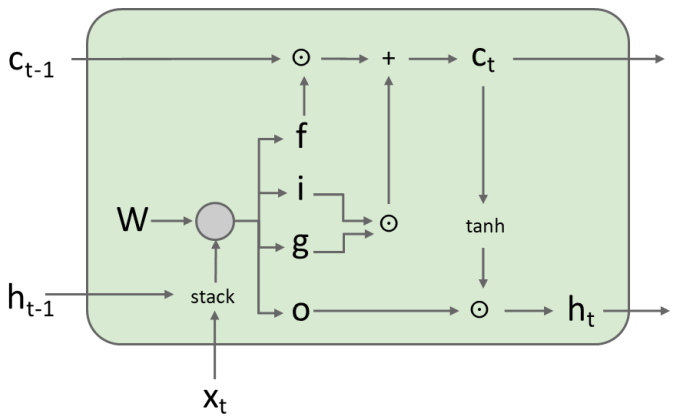
$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

- The LSTM controls information flow into and out of the Cell State using 'Gates'.
- A gate acts as a mathematical valve.
- It uses a Sigmoid activation function to squash values into [0, 1].
  - 0.0 = completely blocked (forget).
  - 1.0 = completely open (remember).

Regulating Information with Gates



$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

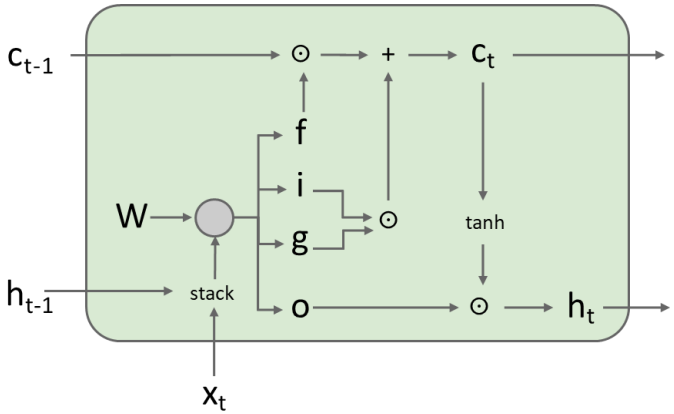
$$h_t = o \odot \tanh(c_t)$$

- Equation:

$$\boldsymbol{Gate} = \boldsymbol{\sigma}\left(\boldsymbol{W} * [\boldsymbol{h_{t-1}}, \boldsymbol{x_t}]\right)$$

- The gate vector is multiplied element-wise with the target vector (see the $\odot$ in the diagram)
- An LSTM uses three distinct gates: Forget (f), Input (i), and Output (o).

LSTM Step 1: The Forget Gate



$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

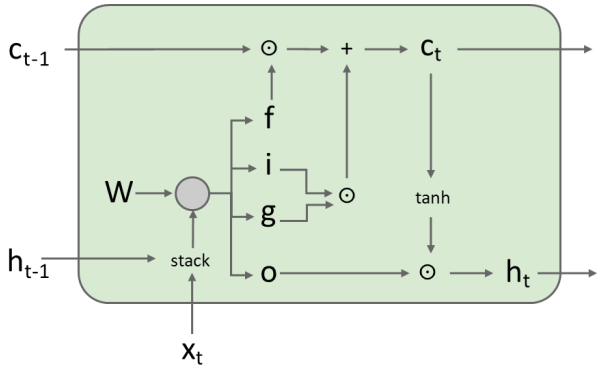
$$h_t = o \odot \tanh(c_t)$$

- Goal: Decide what information in the long-term memory ($\boldsymbol{C_{t-1}}$) should be erased.

$$f_t = \sigma\left(W_f * [h_{t-1}, x_t]\right)$$

- Example: If the subject changes from 'Bob' to 'Alice', the Forget Gate outputs 0s to erase Bob's masculine pronouns from the Cell State.

LSTM Step 2: The Input Gate & g Gate



$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

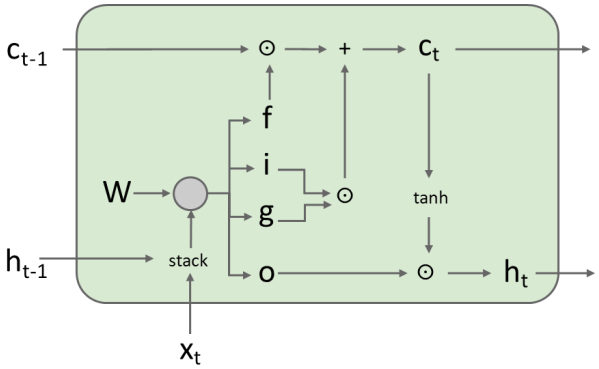
- Goal: Decide what NEW information to store in the Cell State.
- Part A: g Gate (The new data).

$$g_t = tanh\left(W_g * [h_{t-1}, x_t]\right)$$

- Part B: Input Gate (What percentage of data to keep).

$$i_t = \sigma\left(W_i * [h_{t-1}, x_t]\right)$$

LSTM Step 3: Updating the Cell State



$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$\boxed{c_t = f \odot c_{t-1} + i \odot g}$$

$$h_t = o \odot \tanh(c_t)$$

- Now, we compute the new Long-Term Memory (C_t).

$$C_t = f_t \odot C_{t-1} + i_t \odot g_t$$

The $\odot$ denotes element-wise multiplication

- Notice the ADDITION operator (+). We take the old memory, erase parts, and ADD new memory.

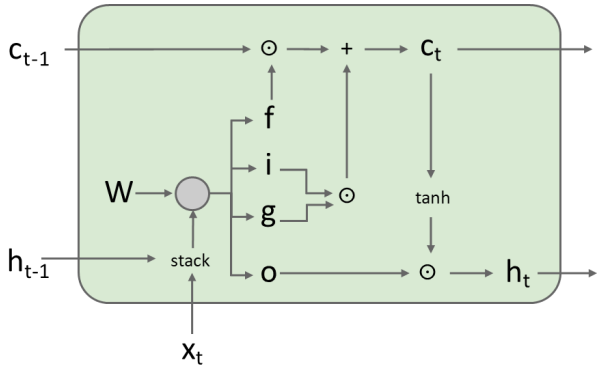
The Mathematical Magic: Addition

- This is the most critical math in the LSTM: $C_t = \ldots + \ldots$
- Because the operation is ADDITIVE, the backward pass derivative distributes the gradient equally.
- We have bypassed the repeated matrix multiplication curse.

The Constant Error Carousel (CEC)

- This mechanism is called the Constant Error Carousel.
- Error signals trapped in the cell state can revolve indefinitely without decaying or exploding.
- This enables LSTMs to track dependencies across thousands of steps.

LSTM Step 4: The Output Gate



$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

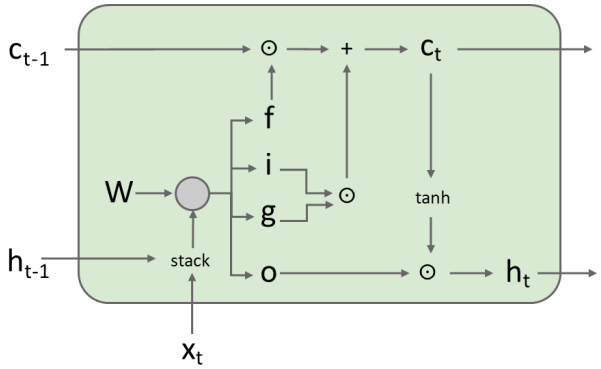
$$h_t = o \odot \tanh(c_t)$$

- Goal: Decide what to output as the new Hidden State (Short-Term Memory).

$$o_t = \sigma(W_o * [h_{t-1}, x_t])$$

- We do not want to output the entire long-term memory at once. The Output gate filters it.

LSTM Step 5: Final Hidden State



$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$\boxed{h_t = o \odot \tanh(c_t)}$$

$$h_t = o_t * tanh(C_t)$$

- We push the Cell State through $tanh$ (normalizing to $[-1, 1]$) and multiply by the Output Gate
- This h_t is used to predict the next token, and is passed as the short-term memory to next cell.

LSTM Parameter Complexity

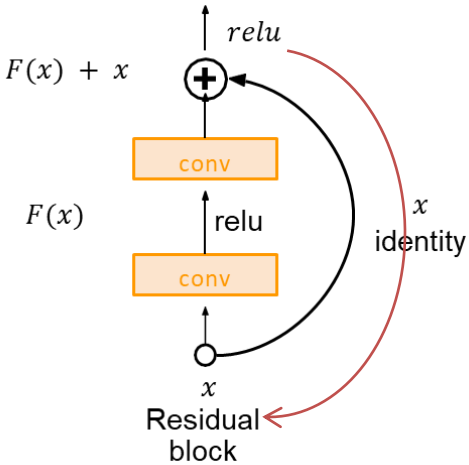
- An LSTM is powerful, but computationally heavy.
- A vanilla RNN uses one weight matrix W_{hh}.
- An LSTM uses FOUR independent sets of matrices (Forget, Input, g , Output).
- Parameter count and memory footprint are roughly 4x larger.

LSTM and ResNet

- The ResNet Highway
  - ResNets solve vanishing gradients in deep networks using Skip Connections

$$H(x) = F(x) + x.$$

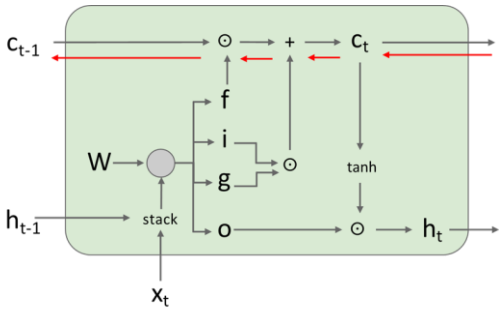
  - The additive $+x$ creates an identity highway where the gradient is exactly 1, allowing gradients to flow backward through many layers unaltered.



- The LSTM Conveyor Belt (Sequence Modeling)
  - LSTMs use a remarkably similar additive mechanism for their cell state

$$C_t = f_t \odot C_{t-1} + i_t \odot g_t.$$

  - Instead of a static "+1" connection, the LSTM uses a Dynamic Skip Connection.
  - When the forget gate $f_t \approx 1$, the cell state acts like an identity function ($C_t \approx C_{t-1}$).
  - This mimics a ResNet skip connection, preventing gradients from vanishing over long sequences.



Do LSTMs solve the vanishing gradient?

- The LSTM architecture makes it easier for the RNN to preserve information over many timesteps
  - e.g. if the f = 1 and the i = 0, then the information of that cell is preserved indefinitely.
  - By contrast, it's harder for vanilla RNN to learn a recurrent weight matrix W_h that preserves info in hidden state

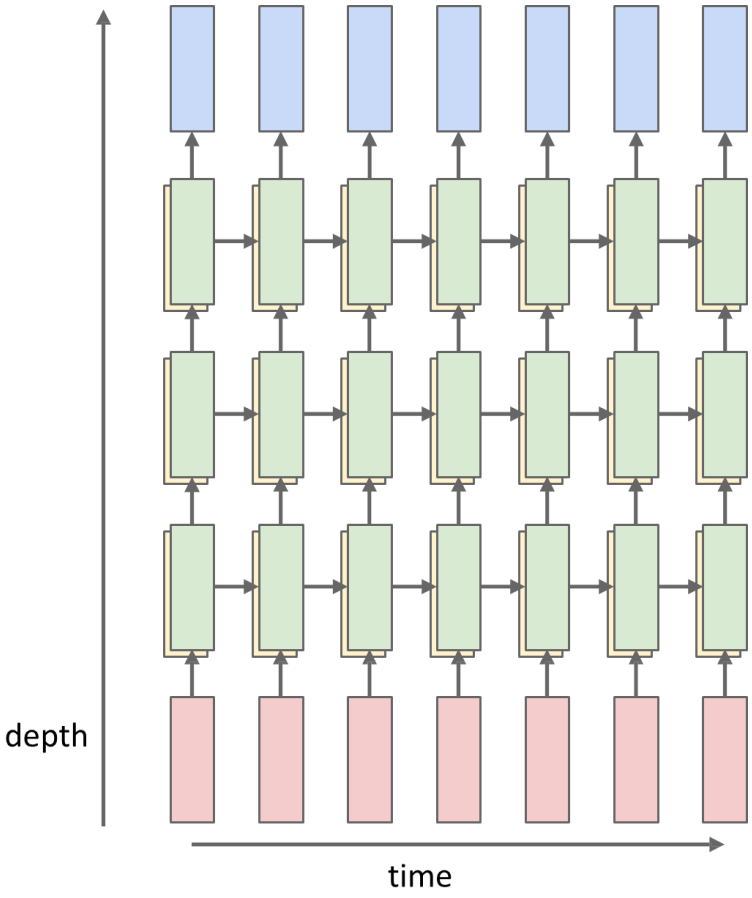
- LSTM doesn't guarantee that there is no vanishing/exploding gradient, but it does provide an easier way for the model to learn long-distance dependencies

Modern RNNs

- Gated Recurrent Units (GRU) was Introduced by Cho et al. in 2014 as a computationally lighter alternative to LSTM.
  - Motivation: Retain the additive gradient highway, but reduce parameters.
  - GRUs merge the Cell State and Hidden State into one unified vector h_t.

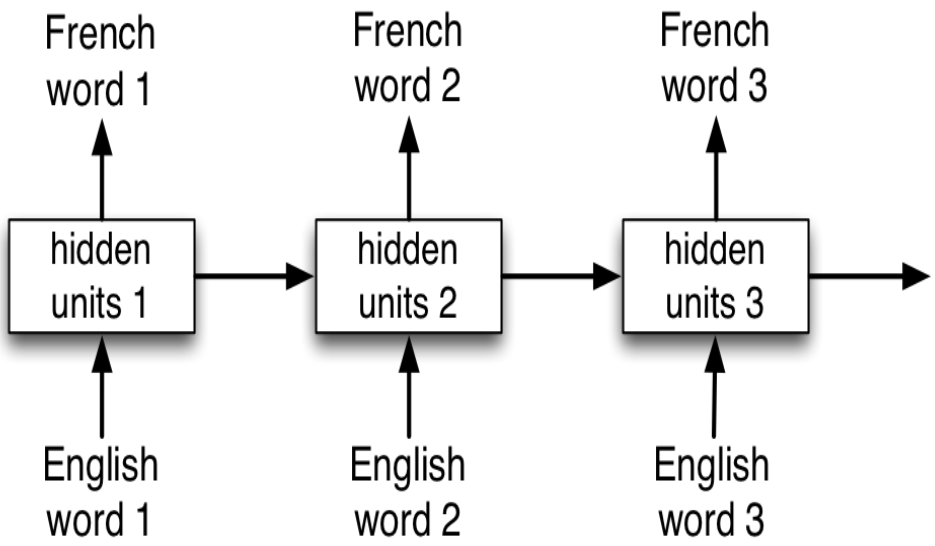
Multi-Layer (Deep) RNNs

- RNNs can be stacked vertically (depth).
- The hidden state sequence from Layer 1 acts as the input sequence to Layer 2.
- Allows the network to compute highly abstract hierarchical representations (e.g., Layer 1 = Syntax, Layer 3 = Semantics).
- Usually capped at 2 to 4 layers; deeper networks become unstable.



Neural Machine Translation

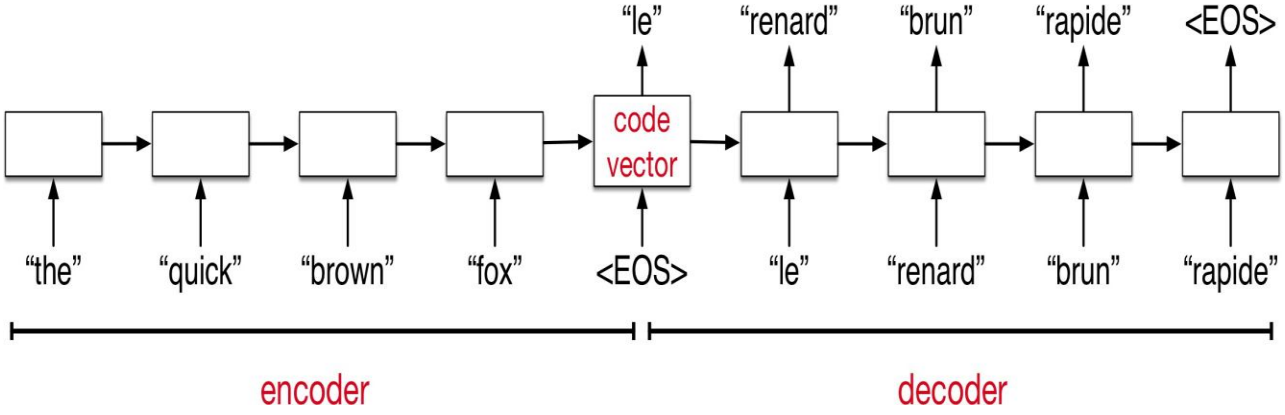
- We'd like to translate, e.g., English to French sentences, and we have pairs of translated sentences to train on.
- What's wrong with the following setup?



- The sentences might not be the same length, and the words might not align perfectly.
- You might need to resolve ambiguities using information from later in the sentence.

Neural Machine Translation

- Sequence-to-sequence architecture: the network first reads and memorizes the sentence. When it sees the end token, it starts outputting the translation.



- The encoder and decoder are two different networks with different weights
- Encoder: An LSTM reads the English sentence and compresses it into a final hidden state h_T.
- Decoder: A separate LSTM takes h_T as its initial state and generates French word-by-word.
- This dominated Google Translate from 2014-2017.

- *Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation*, K. Cho, B. van Merrienboer,C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, Y. Bengio. EMNLP 2014.
- *Sequence to Sequence Learning with Neural Networks*, Ilya Sutskever, Oriol Vinyals and Quoc Le, NIPS 2014.

LSTM vs Transformers

Transformers vs LSTMs

- Are LSTMs Dead?

No

- Transformers require $O(N^2)$ memory relative to the sequence length N.
- For infinitely long, continuous data streams (e.g., IoT sensor data, heart rate monitors), Transformers run out of RAM.
- LSTMs use $O(1)$ memory regarding sequence length, maintaining their state indefinitely.
  - Because an LSTM processes data step-by-step and summarizes the history into a fixed-size Hidden State (h_t) and Cell State (C_t), the amount of RAM it uses stays the same whether it has processed 10 tokens or 10 million tokens.
- LSTMs remain standard in embedded edge devices.

Next Time

- **Attention**
- **Transformers**